Skeleton-Aware Graph Embeddings for Multi-Person Pose Grouping

by

Dean Spencer Mills

Submitted in partial fulfillment of the requirements for the degree

Master of Engineering (Computer Engineering)

in the

Department of Electrical, Electronic and Computer Engineering

Faculty of Engineering, Built Environment and Information Technology

UNIVERSITY OF PRETORIA

January 2025

SUMMARY

Skeleton-Aware Graph Embeddings for Multi-Person Pose Grouping

by

Dean Spencer Mills

Supervisor:	<to be completed>
Co-supervisor:	<to be completed> [optional line]
Department:	Electrical, Electronic and Computer Engineering
University:	University of Pretoria
Degree:	Master of Engineering (Computer Engineering)
Keywords:	Keyword, keyword, keyword, keyword, keyword, keyword, keyword, keyword, keyword, keyword, keyword, keyword

This is the EECE dissertation/thesis template for Latex, version 2025-11-24. Summary paragraph one summary paragraph one.

Summary paragraph two summary paragraph two summary paragraph two summary paragraph two summary paragraph two summary paragraph two summary paragraph two summary paragraph two summary paragraph two summary paragraph two summary paragraph two summary paragraph two summary paragraph two summary paragraph two summary paragraph two summary paragraph two

summary paragraph two summary paragraph two summary paragraph two summary paragraph two summary paragraph two summary paragraph two.

Summary paragraph three summary paragraph three summary paragraph three summary paragraph three summary paragraph three summary paragraph three summary paragraph three summary paragraph three summary paragraph three summary paragraph three summary paragraph three summary paragraph three summary paragraph three summary paragraph three.

LIST OF ABBREVIATIONS

ANN	artificial neural network
EMI	electromagnetic interference
MCMC	Markov chain Monte Carlo
SVM	support vector machine
Note:	Make sure the list of abbreviations is sorted alphabetically. This list is formatted as a table with no borders. Insert rows as needed.

LIST OF SYMBOLS [OPTIONAL]

I	current
R	resistance
V	voltage
Note:	Make sure the list of symbols is sorted alphabetically, and/or according to the Greek alphabet. The list is formatted as a table with no borders. Insert rows as needed.

TABLE OF CONTENTS

LIST OF FIGURES

[THE LIST OF FIGURES IS OPTIONAL]

LIST OF TABLES

[THE LIST OF TABLES IS OPTIONAL]

CHAPTER 1 INTRODUCTION

This chapter motivates multi-person pose grouping as a separable problem within bottom-up human pose estimation (HPE), identifies the gap in modular detector-agnostic grouping methods, and sets out the research questions, approach, and contributions that the rest of the dissertation defends.

1.1 CONTEXT

Introduces the detection/grouping split in bottom-up HPE: modern detectors (HigherHRNet, HRNet) produce accurate per-joint heatmaps; assigning those joints to person identities is the remaining bottleneck. Summarises why joint-to-person assignment is intrinsically difficult (unknown K, type constraints, occlusion, scale) and why it deserves to be studied as a module in its own right.

1.2 PROBLEM STATEMENT

Formulates the task: given N detected keypoints with joint-type labels $t_i \in \{1, \ldots, 17\}$ from an image containing an unknown number of people K, assign each keypoint to one of K person groups under the constraint that every person has at most one keypoint of each type.

1.3 RESEARCH GAP

Existing grouping methods (associative embeddings in HigherHRNet, part-affinity fields in OpenPose, HGG, CenterGroup) are tightly coupled to their host detectors and trained jointly with them. A *modular* grouping module that operates on any detector's keypoint output is largely unstudied. This is the gap this work addresses.

1.4 RESEARCH OBJECTIVE AND RESEARCH QUESTIONS

The objective is to design and evaluate a detector-independent, GNN-based pose grouping module. The investigation addresses the following research questions. Bracketed short answers anticipate the findings from Chapter 4; full support is provided there.

- **RQ1 — The lever.** Is the bottleneck for clustering-based pose grouping the embedding, or the grouping algorithm? *[Answer: the embedding. Four learned grouping heads (DEC, slot attention, graph partitioning, SA-DMoN) fail to beat plain kNN on the same embeddings; strengthening the embedding (PG-GAT) then beats every prior configuration.]*
- **RQ2 — Inductive biases.** Which skeleton-aware modifications to a GATv2 embedding improve pose grouping, and by how much? *[Answer: three modifications — type-pair attention modulation, skeleton-aware positional encoding, and a type-repulsion loss term — each contribute independently; the full combination is the best configuration.]*
- **RQ3 — Detector independence.** Can a grouping module trained separately from the detector match an integrated end-to-end grouping mechanism (HigherHRNet's associative embedding head) on a real-image benchmark? *[Answer: not fully. PG-GAT + kNN end-to-end is approximately 0.04 PGA below HigherHRNet AE on COCO, with detector-independence as the trade-off.]*
- **RQ4 — Embedding geometry.** Does the choice of embedding space (Euclidean vs hyperbolic) affect pose grouping quality? *[Answer: yes before COCO fine-tuning, where hyperbolic at $d = 32$ matches Euclidean at $d = 128$ (an efficiency edge). After fine-tuning the advantage disappears — reported as an efficiency trade, not a quality win.]*
- **RQ5 — Graph primitive.** Is a higher-order graph primitive (skeleton triplets) a better node for pose grouping than single joints? *[Answer: no. TriGAT underperforms joint-level PG-GAT on every comparison, with the voting step (triplet clusters to joint labels) as the dominant failure mode.]*
- **RQ6 — Fine-tuning.** How much does COCO fine-tuning lift a synthetically-trained embedding, and does it damage performance on the synthetic domain? *[Answer: +7.5 PGA on COCO (kNN), the single largest gain of any experiment in this work; synthetic performance is not damaged (0.910 → 0.928).]*

1.5 APPROACH

Train a graph attention network with contrastive loss on detected keypoints; cluster at test time with kNN. The iteration order mirrors the investigation: standard GAT baseline → a series of learned grouping heads (DEC, slot attention, graph partitioning), all of which failed to beat kNN → SA-DMoN, a skeleton-aware modularity objective, which also failed → PG-GAT, which keeps the clustering head simple but strengthens the embedding with three skeleton-aware modifications. Evaluate on synthetic

data (controlled K, perfect GT) and COCO val2017 (zero-shot and fine-tuned). Close the loop by integrating with HigherHRNet for end-to-end validation.

1.6 RESEARCH CONTRIBUTIONS

The contributions of this work are:

1. **PG-GAT**, a skeleton-aware graph attention network for pose-keypoint embedding. Three modifications over standard GATv2: type-pair attention modulation, skeleton-aware positional encoding, and a type-repulsion term in the contrastive loss.
2. **Empirical evidence that the embedding, not the grouping head, is the decisive lever** for clustering-based pose grouping. Documented across four learned-head approaches (DEC, slot attention, graph partitioning, and ten SA-DMoN variants), none of which outperformed plain kNN on the same embeddings.
3. **A COCO fine-tuning protocol** that lifts PG-GAT to 0.957 PGA on COCO val2017 with kNN clustering, and end-to-end to 0.942 when paired with HigherHRNet detection.
4. **An alternative-embedding-space study** (hyperbolic PG-GAT in the Lorentz model): hyperbolic at $d = 32$ matches Euclidean at $d = 128$ pre-fine-tune, but the advantage disappears after COCO fine-tuning — documented as an efficiency trade, not a quality win.
5. **An alternative-primitive study** (TriGAT, triplet nodes): a negative result reported with the failure-mode analysis (voting step propagates errors).

1.7 RESEARCH OUTPUTS

The primary research output is a journal paper on PG-GAT, targeted at *Image and Vision Computing* (Q1, IF 5.34), with *Pattern Recognition* (Q1, IF 9.84) as a stretch target.

1.8 OVERVIEW OF STUDY

Chapter 2 reviews the literature on bottom-up human pose estimation, graph neural networks, and clustering methods relevant to pose grouping. Chapter 3 presents the mathematical preliminaries (GNN, GCN, GAT/GATv2, DEC, slot attention, graph partitioning, DMoN) and describes the methodology behind each method investigated, culminating in PG-GAT. Chapter 4 reports experimental results across the methods. Chapter 5 discusses what the results reveal about the problem. Chapter 6 concludes and indicates future work.

1.9 DECLARATION REGARDING USE OF GENERATIVE AI

Please declare if and how generative AI was used in this work. This must be done following the steps (adapted for dissertations/theses) outlined in 8.2.1.B.10 in the IEEE Publication Services and Products Board Operations Manual[1]:

> The use of content generated by artificial intelligence (AI) in . . . [a dissertation/thesis] (including but not limited to text, figures, images, and code) shall be disclosed . . . The AI system used shall be identified, and specific sections of the . . . [dissertation/thesis] that use AI-generated content shall be identified and accompanied by a brief explanation regarding the level at which the AI system was used to generate the content.

While the use of AI systems for editing and grammar enhancement is common practice, its use in this role must also be disclosed.

[1] https://pspb.ieee.org/images/files/PSPB/opsmanual.pdf

CHAPTER 2 LITERATURE REVIEW

This chapter situates multi-person pose grouping within the broader human pose estimation literature, reviews the existing families of grouping mechanisms, and surveys the graph neural network and representation-learning work that PG-GAT builds on. The review closes by identifying the specific gap that the present work addresses.

2.1 MULTI-PERSON HUMAN POSE ESTIMATION

Human pose estimation (HPE) is the problem of localising a fixed set of anatomical keypoints — such as the seventeen joints of the COCO skeleton [1] — on every person present in an image. The dissertation restricts attention to the 2D, multi-person, image-level formulation: per-image keypoint coordinates for an unknown number of people, without temporal context and without 3D reconstruction. Three surveys provide broader scope and historical depth: Chen et al. [2] cover monocular HPE end-to-end, Zheng et al. [3] provide a more recent deep-learning-focused survey, and Andriluka et al. [4] anchor the modern benchmark culture with the MPII dataset.

The single-person estimators that underpin most multi-person systems emerged in a clear lineage of convolutional architectures. Toshev and Szegedy's DeepPose [5] framed keypoint localisation as direct coordinate regression from a deep network. Subsequent architectures shifted to dense heatmap prediction: convolutional pose machines [6] introduced iterative belief-map refinement, and the stacked hourglass network [7] established the encoder–decoder-with-skip-connections topology that underpins most modern single-person estimators. Xiao et al.'s "simple baselines" work [8] showed that a ResNet trunk with three deconvolution layers matched hourglass accuracy at lower complexity, establishing a strong baseline that subsequent methods are still measured against. The HRNet family [9] pushed this further by running several parallel branches at different spatial resolutions throughout the network and exchanging information between them at every stage, rather than recovering resolution through up-sampling at the end of an encoder–decoder pass. The resulting consistent access to high-resolution

feature maps yielded sharper keypoint heatmaps and lower localisation error than any prior backbone of comparable parameter count, which is why HRNet became the de facto backbone for most top-down systems and for the bottom-up HigherHRNet variant [10] discussed in Section 2.2.

2.1.1 Top-Down Approaches

Top-down systems decompose multi-person HPE into two stages: a person detector produces bounding boxes, and a single-person estimator runs on each crop. The canonical instantiations of this architecture are Mask R-CNN [11], which attaches a per-keypoint mask head to the Faster R-CNN detector, and AlphaPose [12], which couples a person-detection front-end with a symmetric spatial transformer and a parametric pose non-maximum suppression step. Top-down methods inherit the strong per-person localisation accuracy of the underlying single-person estimator and consequently dominate the COCO leaderboards across most years. Two structural limitations apply. First, runtime scales linearly with the number of detected people, which becomes prohibitive in dense scenes. Second, performance is bounded by detection quality: missed or over-merged person detections propagate directly to missed or incorrectly grouped keypoints, with no opportunity for the pose estimator to recover.

2.1.2 Bottom-Up Approaches

Bottom-up methods reverse the dependency: all keypoints in the image are detected first, and a separate grouping mechanism assigns each keypoint to a person. Runtime is then approximately constant in the number of people, and detection errors are localised to individual keypoints rather than to entire person instances. Three distinct grouping lineages have emerged within the bottom-up family, and the literature on the grouping mechanisms themselves is reviewed in Section 2.2; the present section names them only to map the bottom-up landscape.

The first lineage, originating with OpenPose [13] and extended by PifPaf [14], predicts a dense 2D vector field along each anatomical limb (the “part affinity field”) in addition to per-keypoint heatmaps. At inference time, candidate keypoint pairs are scored by the line integral of the affinity field between them, and consistent person skeletons are recovered by bipartite matching on the resulting graph. PifPaf adds a Laplace-distributed regression head on top of the same field structure to improve fine-grained localisation in crowded scenes. The second lineage, introduced by Newell et al. as associative embedding [15] and extended to high-resolution backbones by HigherHRNet [10], attaches a learned scalar (or short vector) tag to each detected keypoint and trains the network to make tags of same-person keypoints close and tags of different-person keypoints far apart. Grouping at inference is then a clustering operation on tag values, typically resolved by simple thresholding. The third lineage,

exemplified by DEKR [16] and CenterGroup [17], predicts a person-centre heatmap and per-keypoint offset vectors from those centres, assembling each person skeleton by following the offsets back to the keypoint locations. End-to-end transformer methods (Section 2.1.3) collapse the detect-and-group structure entirely; they are catalogued separately here because they no longer admit a modular grouping stage.

2.1.3 Transformer-Based Approaches

The transformer architecture has been applied to HPE in two distinct ways: as a drop-in replacement for the convolutional backbone of an otherwise standard heatmap-based estimator, and as a complete reformulation of the multi-person task as end-to-end set prediction with no separate grouping stage. The two lines of work have different consequences for whether a modular grouping module of the kind developed here can be attached downstream.

TokenPose [18] demonstrated that a pure-transformer network treating each keypoint as a learned token could match the heatmap accuracy of CNN baselines at comparable parameter counts. HRFormer [19] preserved the parallel-resolution structure of HRNet while replacing convolutions with local-window self-attention, recovering high-resolution feature representations in the transformer family. ViTPose [20] took the opposite design choice — a plain, monolithic Vision Transformer as backbone with no high-resolution path and no domain-specific inductive bias for keypoint localisation — and showed that this minimal architecture nonetheless scales cleanly to model sizes from 100M up to 1B parameters and consistently improves with scale. A simple deconvolution head turns the patch-token grid into per-keypoint heatmaps, so ViTPose plugs into the same downstream pipeline as a CNN backbone. Its successor ViTPose++ [21] adds a mixture-of-experts formulation to cover the wholebody, hand, and animal-pose distributions within a single trunk.

A separate line of transformer work reformulates multi-person HPE as direct set prediction, dispensing with the explicit grouping stage altogether. PETR [22] was the first end-to-end pose detector to apply the DETR set-prediction paradigm to human keypoints: a fixed pool of pose queries attends to the image feature map, each query learns to predict a complete per-person skeleton, and bipartite matching at training time assigns each ground-truth person to one of the queries. There is no separate grouping operation at inference; the binding between keypoints and a person is the query identity. QueryPose [23] sparsified the attention pattern by splitting each pose query into part-level sub-queries. ED-Pose [24] and GroupPose [25] further developed the query-based formulation, reaching competitive performance

on the COCO benchmark without any post-hoc grouping step. These methods couple grouping tightly to the backbone: they do not produce explicit per-keypoint embeddings that a downstream module could group separately, and the grouping mechanism is consequently not transferable to checkpoints from other detectors.

2.1.4 Heatmap and Regression Trends

The traditional bottleneck of regression-based pose estimation is the discontinuity of the argmax operator that maps a continuous output to a discrete keypoint coordinate. Sun et al.'s integral pose regression [26] bridged the heatmap and regression paradigms with a differentiable soft-argmax, and DSNT [27] formalised the same idea as a normalised spatial-coordinate layer. A direct probabilistic treatment was given by residual log-likelihood estimation (RLE) [28], which models the residual between predicted and ground-truth coordinates with a flow-based density and was the first regression method to exceed heatmap accuracy on multi-person COCO. SimCC [29] reformulated coordinate prediction as a two-axis classification problem, producing the underlying mechanism for the real-time RTMPose family [30]. RTMO [31] unifies detection and regression in a single stage, and RTMW [32] extends the family to whole-body pose at real-time rates. These methods are relevant here only insofar as they produce per-keypoint coordinates; the grouping module proposed in Chapter 3 operates on detected coordinates and is agnostic to whether they originated from a heatmap or a regressor.

2.1.5 Foundation Models for Pose Estimation

Recent work has pushed the backbone trajectory toward foundation-model scale. Sapiens [33] pretrains Vision Transformer backbones from 0.3 to 2 billion parameters on 300M curated human images, producing a unified human-vision backbone that reaches state-of-the-art accuracy across pose estimation, segmentation, depth, and surface normal prediction. The same trend toward promptable, dataset-unified methods is represented by X-Pose [34], which detects arbitrary keypoint categories conditioned on a textual or visual prompt across thirteen datasets, and by the compositional-token approach of PCT [35], which represents poses as discrete codebook tokens to improve occlusion robustness. The foundation-model trajectory matters here as context rather than as comparison: a sufficiently large pretrained backbone reduces the per-keypoint localisation error to near saturation, which sharpens the case for studying grouping as a modular problem downstream of detection.

2.1.6 Benchmarks

Three datasets dominate the 2D multi-person literature. MPII Human Pose [4] established the modern benchmark protocol with 25K images and approximately 40K annotated people across 410 activities; its focus is single-person crops and it has been largely superseded for multi-person research. COCO [1]

is the de facto multi-person benchmark, with seventeen keypoints per person and approximately 250K annotated instances across the train, val, and test partitions; val2017 is the standard reporting set and the one used for the COCO evaluations in Chapter 4. CrowdPose [36] targets the dense-crowd regime explicitly, with a crowd-index metric and approximately 20K images selected for inter-person occlusion; it is the appropriate benchmark for stress-testing grouping methods, although it is not used in the experiments that follow. The OCHuman benchmark, introduced alongside Pose2Seg [37], isolates occluded humans and is similarly relevant to grouping robustness.

Across these benchmarks the dominant evaluation metric is average precision (AP) under the Object Keypoint Similarity (OKS) family of thresholds, introduced with the COCO benchmark itself [1]. OKS measures the agreement between a predicted and ground-truth keypoint as a Gaussian-tolerance similarity in which the variance depends on the keypoint type and the size of the person bounding box; a prediction is counted as correct above an OKS threshold, and AP is averaged across thresholds in the range $[0.50, 0.95]$ in steps of 0.05. The earlier MPII-era metric, Percentage of Correct Keypoints (PCK), counts a prediction as correct if it falls within a fixed fraction of the head segment length; it is still occasionally reported on MPII but has been largely superseded by AP@OKS on multi-person datasets. The dissertation does not report AP@OKS directly because its grouping contribution is evaluated as a Hungarian-matched per-joint accuracy (see Section 3.13), which isolates the grouping decision from detection quality; however, every grouping baseline cited in this chapter is reported in its original publication on the AP@OKS metric, and the relationship between the two is discussed in Chapter 4.

2.1.7 Pose Tracking

Multi-person 2D pose tracking extends per-image HPE with temporal association across frames. PoseFlow [38] is the canonical online tracker that propagates per-person identities through pose-similarity matching in a small temporal window. LightTrack [39] provides a top-down tracking framework with a learned re-identification head. The PoseTrack benchmark [40] is the standard evaluation set for this video-pose formulation. Pose tracking is out of scope for the present work, which is restricted to per-image grouping. The grouping decisions are nonetheless the bottleneck within each tracked frame: once per-frame skeletons have been assembled, temporal association is typically a comparatively shallow re-identification problem over short time windows, so the per-image grouping module developed in Chapter 3 is compatible with downstream temporal association via standard ReID techniques.

2.1.8 Position of This Dissertation

The HPE literature has converged on a small set of strong single-image detection backbones whose per-keypoint localisation accuracy is approaching saturation. HRNet-family backbones already reach mean OKS-based average precision above 0.75 on COCO val2017, and recent foundation-model approaches such as Sapiens [33] push that figure higher by pretraining on hundreds of millions of human-centric images. For systems that do not yet use foundation-model-scale backbones, real-time variants such as RTMPose [30] and RTMO [31] compress much of the same accuracy into far smaller models, indicating that per-keypoint localisation is no longer the binding constraint on multi-person performance for typical deployment regimes.

The remaining sources of error in bottom-up pipelines are therefore concentrated downstream of detection, in the grouping decision that assigns each detected keypoint to a person identity. In the established lineages reviewed above, this decision is taken in ways that prevent decoupling from the detector: part-affinity fields are predicted by the same network that produces the heatmaps, associative-embedding tags are trained jointly with the heatmap loss on a specific backbone, and centre-regression methods such as DEKR fold the grouping into a backbone-specific keypoint-from-centre offset head. End-to-end transformer pose detectors take the coupling further still: no per-keypoint representation is ever exposed, because the binding between keypoints and a person is the identity of the pose query that produced them. In all of these cases, swapping out the detector after training is not a supported operation, and grouping research is consequently constrained to architectures whose authors release both the detector weights and the matching grouping head.

The lineage of *modular* grouping methods — methods that take detected keypoints as input from any detector, run a separable algorithm, and return per-keypoint person identities — is much sparser. A handful of recent works can be read in this lineage: HGG [41] formulates grouping as higher-order graph matching on detected keypoints, GEC [42] as binary edge classification, and the instance-aware approach of Yang et al. [43] as a learned per-person assignment head. This dissertation contributes to the same lineage with a graph-neural-network embedding approach (PG-GAT, Chapter 3) that operates on the detected-keypoint output of an arbitrary upstream detector and produces per-keypoint embeddings on which simple clustering recovers person identities. The next two sections develop the supporting literature on grouping mechanisms (Section 2.2) and on graph neural networks for pose (Section 2.3), both of which directly inform the design of the contributed method.

2.2 GROUPING IN BOTTOM-UP POSE ESTIMATION

Review of the major grouping mechanisms in the bottom-up literature: part-affinity fields (OpenPose), associative embeddings (Newell et al.), PersonLab mid-range offsets, HGG (graph-based), CenterGroup. How each binds tightly to its host detector and why that limits modularity.

2.3 GRAPH NEURAL NETWORKS FOR POSE

Existing graph-based approaches in the pose literature: STGAT (skeleton-based action recognition), GAST-Net (single-person 3D pose from video), skeleton-aware GCN variants. How these differ from the multi-person keypoint grouping task — none of the existing "skeleton-aware" networks target grouping, which motivates both the PG-GAT contribution and the naming choice (avoiding collision with STGAT/GAST-Net descriptors).

2.4 REPRESENTATION LEARNING FOR CLUSTERING

Contrastive learning (InfoNCE, SimCLR), triplet losses, metric learning. Deep clustering lineage: DEC (Xie et al.), IDEC, DeepCluster, DMoN. What these methods assume about the feature space and how those assumptions interact with the pose-grouping task.

2.5 GEOMETRIC EMBEDDING SPACES

Euclidean versus non-Euclidean embedding spaces. Hyperbolic embeddings (Poincaré ball, Lorentz model) and their suitability for tree-structured data. Relevance to skeletons, which are trees. Sets up the motivation for the hyperbolic PG-GAT variant evaluated in Chapter 4.

2.6 RESEARCH GAP

Synthesises the review: modular grouping modules for bottom-up HPE are under-explored; existing graph-based pose networks target different tasks; the clustering-head literature has not been systematically applied to pose grouping; and the choice of embedding space (Euclidean vs hyperbolic) is untested in this domain. PG-GAT and its ablations address these gaps.

CHAPTER 3 METHODOLOGY

This chapter develops the methodology for every result reported in the rest of the work. It opens with two preliminary sections that fix the mathematical vocabulary the chapter relies on: graph neural networks (Section 3.1) up to the GATv2 attention primitive used throughout, and clustering and learned grouping heads (Section 3.2) covering k-means, DEC, slot attention, graph partitioning, and DMoN. The problem formulation, dataset pipeline, and baseline (Sections 3.3–3.5) then state the task and the comparison point that every method clears or falls short of. The next four sections trace the investigation in the order it actually unfolded: three learned grouping heads (Section 3.6) and the SA-DMoN modularity family (Section 3.7) both fail to outperform k-means on the baseline embedding, which motivates the shift to modifying the embedding network itself and the centrepiece PG-GAT design of Section 3.8. Three independent variants of PG-GAT — a visual-feature augmentation (Section 3.9), a hyperbolic-geometry variant (Section 3.10), and a triplet-based variant (Section 3.11) — stress-test the design from different angles. The chapter closes with the integration of PG-GAT into a realistic end-to-end pipeline (Section 3.12) and the evaluation protocol and hyperparameter-defence methodology (Section 3.13) that every Chapter 4 number is reproducible from.

3.1 PRELIMINARIES — GRAPH NEURAL NETWORKS

A human skeleton is a graph: seventeen keypoints form the nodes and the skeletal connections form the edges. A graph neural network (GNN) embeds each node while conditioning on the structure of the graph — the natural choice for producing pose-grouping embeddings that are aware of joint-type and connectivity information. This section introduces the message-passing framework that unifies modern GNN variants, derives the graph convolutional network (GCN) as the canonical spectral instance, and develops the graph attention network (GAT), culminating in the GATv2 correction that is used as the attention primitive in every experiment in this chapter.

3.1.1 The Message-Passing Framework

Let $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ denote an undirected graph with N nodes and edge set $\mathcal{E} \subseteq \mathcal{V} \times \mathcal{V}$. Each node $i \in \mathcal{V}$ is associated with a feature vector $\mathbf{h}_i^{(0)} \in \mathbb{R}^{d_0}$, and each edge $(i, j) \in \mathcal{E}$ optionally carries an edge feature $\mathbf{e}_{ij}$. The neighbourhood of node i is $\mathcal{N}(i) = \{j : (i, j) \in \mathcal{E}\}$.

Almost every modern GNN variant can be expressed as iterated message passing [44]. At layer $l+1$, each node receives a message from each of its neighbours, aggregates them with a permutation-invariant operator, and updates its representation accordingly. Formally,

$$\mathbf{m}_{i \leftarrow j}^{(l+1)} = \mathrm{MSG}^{(l)}\left(\mathbf{h}_i^{(l)}, \mathbf{h}_j^{(l)}, \mathbf{e}_{ij}\right), \tag{3.1}$$

$$\mathbf{h}_i^{(l+1)} = \mathrm{UPDATE}^{(l)}\left(\mathbf{h}_i^{(l)}, \bigoplus_{j \in \mathcal{N}(i)} \mathbf{m}_{i \leftarrow j}^{(l+1)}\right), \tag{3.2}$$

where $\bigoplus$ denotes a permutation-invariant aggregator (sum, mean, or max) and $\mathrm{MSG}^{(l)}$ and $\mathrm{UPDATE}^{(l)}$ are learnable functions at layer l. Different choices of these three components recover different GNN architectures. GCN (Section 3.1.2) uses a linear message, a degree-based weighted sum aggregator, and a linear–nonlinear update. GAT (Section 3.1.3) introduces learned attention weights on the aggregator, making each neighbour's contribution data-dependent. GraphSAGE [45] uses concatenated mean–max aggregation with a feedforward update, optimised for inductive settings.

The message-passing view clarifies two points that matter for the remainder of this chapter. First, the inductive biases of a GNN are encoded in the choice of aggregator and the edge-weighting scheme, not in the overall template. Second, the number of layers controls the receptive field — a node's representation after L layers aggregates information from its L-hop neighbourhood — which, for skeleton graphs with at most four hops between any two joints, places a natural upper bound on depth.

3.1.2 Graph Convolutional Networks (GCN)

The GCN architecture of [46] derives from a first-order approximation of spectral graph convolutions. Given adjacency matrix $A \in \mathbb{R}^{N \times N}$ and degree matrix $D = \mathrm{diag}(\sum_j A_{ij})$, the symmetric normalised adjacency with self-loops is

$$\tilde{A} = \tilde{D}^{-1/2}(A + I)\tilde{D}^{-1/2}, \tag{3.3}$$

where $\tilde{D} = \mathrm{diag}(\sum_j (A + I)_{ij})$ is the degree matrix of the self-loop-augmented graph. The GCN propagation rule is

$$H^{(l+1)} = \sigma\left(\tilde{A} H^{(l)} W^{(l)}\right), \tag{3.4}$$

where $H^{(l)} \in \mathbb{R}^{N \times d_l}$ stacks the node features row-wise, $W^{(l)} \in \mathbb{R}^{d_l \times d_{l+1}}$ is a learnable weight matrix, and $\sigma(\cdot)$ is a pointwise nonlinearity.

Two properties of Equation (3.4) motivate the move to attention-based aggregation in the pose-grouping setting. First, the edge weights are fixed: once $\tilde{A}$ is constructed from the graph topology, no learning signal modulates the contribution of individual neighbours. Two neighbours of the same target node contribute in proportion determined only by the degrees of the three nodes involved. Second, this uniform treatment is informationally inadequate for skeleton graphs where edge semantics vary sharply: the edge from the left shoulder to the left elbow encodes different information from the edge from the left shoulder to the head, and a pose-grouping embedding should be free to weight them differently. Attention-based aggregation, introduced next, provides exactly this flexibility.

3.1.3 Graph Attention Networks (GAT and GATv2)

GAT [47] replaces the fixed normalisation of Equation (3.3) with a learned attention weight for each edge. For a pair (i, j) with $j \in \mathcal{N}(i)$, the unnormalised attention score is

$$e_{ij}^{\mathrm{GAT}} = \mathrm{LeakyReLU}\Big(\mathbf{a}^{\top}\,[W\mathbf{h}_i \,\|\, W\mathbf{h}_j]\Big), \tag{3.5}$$

where $W \in \mathbb{R}^{d' \times d}$ is a shared linear projection, $\mathbf{a} \in \mathbb{R}^{2d'}$ is a learned attention vector, and $[\cdot\|\cdot]$ denotes vector concatenation. The scores are normalised over the neighbourhood with a softmax,

$$\alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{k \in \mathcal{N}(i)} \exp(e_{ik})}, \tag{3.6}$$

and the updated node representation is the attention-weighted sum of projected neighbour features,

$$\mathbf{h}_i' = \sigma\left(\sum_{j \in \mathcal{N}(i)} \alpha_{ij} W\mathbf{h}_j\right). \tag{3.7}$$

Multi-head attention is recovered by repeating Equations (3.5)–(3.7) with independent parameters for each head and concatenating or averaging the outputs.

Equation (3.5) exposes a subtle limitation that was identified by [48] and has direct consequences for pose grouping. Because $\mathbf{a}^{\top}$ is applied to the concatenation $[W\mathbf{h}_i \| W\mathbf{h}_j]$, the attention score decomposes as $\mathbf{a}_1^{\top} W\mathbf{h}_i + \mathbf{a}_2^{\top} W\mathbf{h}_j$ where $\mathbf{a} = [\mathbf{a}_1; \mathbf{a}_2]$. The term $\mathbf{a}_1^{\top} W\mathbf{h}_i$ is independent of j, so the relative ordering of neighbours by attention score is the same for every query node — a property that [48] formalises as *static attention*. In other words, if node j_1 is ranked above node j_2 as a neighbour of i, then j_1 is also ranked above j_2 as a neighbour of every other node. For a pose-grouping network this is undesirable: the network should be able to judge that a given left shoulder is important to a right shoulder but not to a left ankle of a different person.

GATv2 resolves the limitation by reordering the operations inside the attention score,

$$e_{ij}^{\mathrm{GATv2}} = \mathbf{a}^{\top}\,\mathrm{LeakyReLU}(W\,[\mathbf{h}_i \,\|\, \mathbf{h}_j])\,. \tag{3.8}$$

With the nonlinearity now placed inside the query–key interaction, the attention score does not decompose into independent per-node terms, and the ranking of neighbours is allowed to depend on the query node. GATv2 is strictly more expressive than GAT at the same parameter count [48], and every architecture evaluated in this chapter uses GATv2 as its attention primitive. The remaining GCN and GAT references in Chapters 4 and 5 refer to baselines and background, not to the working network.

3.2 PRELIMINARIES — CLUSTERING AND LEARNED GROUPING

Pose grouping reduces, once an embedding is fixed, to a clustering problem on a finite set of points in an embedding space. This section introduces the clustering primitives that the rest of the chapter builds on: k-means as the canonical baseline and the read-out used throughout this chapter, constrained k-means as the family that admits structural side-information, deep embedded clustering as the unsupervised representation-and-cluster joint learner, slot attention as the supervised iterative competitor, graph partitioning as the affinity-based community detector, and Newman modularity and its differentiable relaxation DMoN as the modularity-driven alternative. Each primitive is defined formally below so that subsequent sections can apply them to pose embeddings without re-deriving their mechanics.

3.2.1 k-means and constrained k-means

Given a set of points $\mathcal{Z} = \{\mathbf{z}_1, \ldots, \mathbf{z}_N\} \subset \mathbb{R}^d$ and a target cluster count k, k-means [49] seeks centres $\boldsymbol{\mu}_1, \ldots, \boldsymbol{\mu}_k \in \mathbb{R}^d$ and an assignment function $c : \{1, \ldots, N\} \to \{1, \ldots, k\}$ that minimise the sum of squared within-cluster distances,

$$\mathcal{L}_{k\text{-means}}(\{\boldsymbol{\mu}_j\}, c) = \sum_{i=1}^{N} \|\mathbf{z}_i - \boldsymbol{\mu}_{c(i)}\|_2^2. \tag{3.9}$$

The standard solver is Lloyd's algorithm [50]: alternate between an assignment step that maps each $\mathbf{z}_i$ to the nearest current centre and an update step that sets each centre to the mean of the points currently assigned to it. The algorithm converges to a local minimum of Equation (3.9) in a finite number of iterations but is sensitive to the initial choice of centres; the k-means++ scheme of [51] chooses initial centres with probability proportional to their squared distance from the nearest already-chosen centre, which gives a $\mathcal{O}(\log k)$-approximation in expectation and stabilises the downstream solution.

k-means is used throughout as both an evaluation read-out and a comparison point. After every embedding method (baseline GAT, PG-GAT, hyperbolic PG-GAT, TriGAT) the per-image partition

is recovered by running k-means with $k = K$ on the L2-normalised embeddings, where K is either provided by an oracle or estimated from the embeddings as described in Section 3.12.3. The L2 normalisation places every embedding on the unit hypersphere, which makes the squared-distance objective of Equation (3.9) equivalent up to an additive constant to a cosine-similarity objective and aligns the clustering metric with the contrastive loss used at training time.

Constrained k-means [52] extends Lloyd's algorithm with pairwise constraints: a *must-link* constraint on $(\mathbf{z}_i, \mathbf{z}_j)$ requires that both points share a cluster, and a *cannot-link* constraint requires that they do not. The assignment step is replaced by a greedy procedure that respects the constraints if a feasible assignment exists. For pose grouping the natural constraint is a cannot-link between every pair of detections of the same joint type: two left elbows in the same image cannot belong to the same person. Constrained k-means and the related structured-assignment methods are introduced here for terminology and are treated in detail in separate companion work referenced where direct comparison is useful; the headline evaluations in this chapter use unconstrained k-means as the read-out.

3.2.2 Deep embedded clustering (DEC)

DEC [53] couples representation learning and clustering into a single differentiable objective. For a fixed cluster count k, DEC maintains a set of cluster centres $\{\boldsymbol{\mu}_j\}_{j=1}^{k} \subset \mathbb{R}^d$ as learnable parameters in the same space as the input embeddings. Each embedding $\mathbf{z}_i$ is assigned to clusters by a Student's t-distribution soft assignment with one degree of freedom [54],

$$q_{ij} = \frac{\left(1 + \|\mathbf{z}_i - \boldsymbol{\mu}_j\|_2^2\right)^{-1}}{\sum_{j'} \left(1 + \|\mathbf{z}_i - \boldsymbol{\mu}_{j'}\|_2^2\right)^{-1}}, \tag{3.10}$$

where q_{ij} is the probability that $\mathbf{z}_i$ belongs to cluster j. A sharpened target distribution p_{ij} is constructed from q_{ij} by raising the probabilities to a power and renormalising,

$$p_{ij} = \frac{q_{ij}^2 / f_j}{\sum_{j'} q_{ij'}^2 / f_{j'}}, \quad f_j = \sum_i q_{ij}, \tag{3.11}$$

which down-weights clusters with high mass and up-weights confident assignments. The training objective is the Kullback–Leibler divergence between the target and the soft assignment, $\mathcal{L}_{\mathrm{DEC}} = \sum_{i,j} p_{ij} \log(p_{ij}/q_{ij})$, and the gradient is propagated through q_{ij} into both the cluster centres and any upstream embedding network. DEC was originally proposed as a fully unsupervised method: no ground-truth label is consumed by the loss, and the only training signal is the self-distillation of the soft assignment against its own sharpened target. Section 3.6.1 adapts this primitive to pose embeddings and discusses why the unsupervised regime is structurally inadequate for the multi-person pose grouping setting.

3.2.3 Slot attention

Slot attention [55] is an iterative competitive attention mechanism originally proposed for unsupervised object-centric representation learning. Given a set of N input feature vectors $\{\mathbf{z}_i\}_{i=1}^{N} \subset \mathbb{R}^d$ and a fixed number of slots K, K slot vectors $\mathbf{s}_1, \ldots, \mathbf{s}_K \in \mathbb{R}^d$ are initialised by sampling from a learned Gaussian distribution $\mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\sigma}^2\mathbf{I})$ whose parameters are trained end-to-end with the rest of the network. At each of T iterations, layer-normalised slots produce queries $\mathbf{q}_j = \mathbf{W}_q\mathbf{s}_j$, layer-normalised inputs produce keys $\mathbf{k}_i = \mathbf{W}_k\mathbf{z}_i$ and values $\mathbf{v}_i = \mathbf{W}_v\mathbf{z}_i$, and the attention matrix is computed by softmax across slots rather than across inputs,

$$A_{ji} = \frac{\exp\left(\mathbf{q}_j^\top \mathbf{k}_i/\sqrt{d}\right)}{\sum_{j'} \exp\left(\mathbf{q}_{j'}^\top \mathbf{k}_i/\sqrt{d}\right)}. \tag{3.12}$$

The cross-slot softmax in the denominator enforces *competition*: each input is contested by every slot, and a stronger query–key alignment in one slot suppresses the contribution of the input to every other slot. The matrix is row-normalised over inputs and used to weight the values into per-slot updates, which a gated recurrent unit (GRU) [56] combines with the previous slot state. A GRU is a small recurrent cell that learns a per-slot reset and update gate so that informative updates accumulate across iterations and noisy updates are suppressed; in the slot-attention setting it acts as a stabiliser that prevents the slots from collapsing onto a single input cluster early in the iteration. After T iterations the final slots produce per-input–per-slot logits by inner product, and the argmax over slots gives a hard assignment.

The slot-attention head is supervised here via Hungarian-matched cross-entropy (Section 3.6.2) rather than through the unsupervised reconstruction loss of the original paper. This is a substantive adaptation: the original slot-attention architecture relies on a downstream decoder that reconstructs the input image, which is absent here. Replacing the reconstruction signal with a supervised assignment signal preserves the competitive attention mechanism while connecting the iteration directly to the ground-truth person labels.

3.2.4 Graph partitioning by edge classification

A graph partitioning approach to clustering does not parameterise the partition directly. Instead, for every pair of nodes (i, j) a binary classifier predicts the logit of the event "i and j belong to the same cluster", and the partition is recovered post-hoc from the connected components of the thresholded affinity graph. The classical objective is the normalised cut [57], which seeks a partition $(\mathcal{V}_1, \ldots, \mathcal{V}_K)$

of the node set that minimises

$$\mathrm{NCut}(\mathcal{V}_1,\ldots,\mathcal{V}_K)=\sum_{k=1}^{K}\frac{\mathrm{cut}(\mathcal{V}_k,\overline{\mathcal{V}_k})}{\mathrm{vol}(\mathcal{V}_k)}, \tag{3.13}$$

where $\mathrm{cut}(\mathcal{V}_k,\overline{\mathcal{V}_k})$ is the sum of edge weights crossing the boundary of $\mathcal{V}_k$ and $\mathrm{vol}(\mathcal{V}_k)=\sum_{i\in\mathcal{V}_k}d_i$ is the volume of the cluster. The normalised cut admits a spectral relaxation [58] whose continuous solution can be read off the second-smallest eigenvalue of the graph Laplacian, but solutions in the discrete combinatorial case are NP-hard.

The learned variant adopted here [59] replaces the spectral relaxation with a learned pairwise affinity score. A pair feature is constructed from the input embeddings $\mathbf{z}_i$ and $\mathbf{z}_j$ as the concatenation of the difference and the element-wise product, which encodes both relative direction and co-activation magnitude in embedding space. A multi-layer perceptron maps the pair feature to a scalar logit, and the network is trained with binary cross-entropy against a ground-truth same/different-person label. At inference the affinity logits are thresholded to a binary affinity graph, and the partition is read off by union–find connected components. Section 3.6.3 applies this primitive to the pose graph and notes the structural property that distinguishes it from slot attention and DEC: the cluster count K does not need to be provided to the partition step, because it falls out as the number of connected components.

3.2.5 Modularity and deep modularity networks (DMoN)

The clustering primitives above all consume embeddings. A different family operates directly on a graph adjacency: given $\mathcal{G}=(\mathcal{V},\mathcal{E})$ with adjacency $A\in\{0,1\}^{N\times N}$ and degrees $d_i=\sum_j A_{ij}$, Newman modularity [60] measures the excess intra-cluster edge mass relative to a chance baseline. For a partition encoded by indicator matrix $\mathbf{S}\in\{0,1\}^{N\times K}$ with $S_{ik}=1$ if node i is in cluster k, modularity is

$$Q(\mathbf{S})=\frac{1}{2m}\mathrm{Tr}\big(\mathbf{S}^\top\mathbf{B}\mathbf{S}\big),\quad \mathbf{B}=\mathbf{A}-\frac{\mathbf{d}\mathbf{d}^\top}{2m}, \tag{3.14}$$

where $m=\frac{1}{2}\sum_{ij}A_{ij}$ is the total edge count and $\mathbf{d}\mathbf{d}^\top/(2m)$ is the *null model* that gives the expected edge count between i and j under a random graph that preserves the degree sequence. Communities are partitions whose intra-cluster edge mass exceeds the null-model prediction, and modularity maximisation seeks the partition that maximises this excess. The construction of the null model is the load-bearing modelling choice: the degree-based null assumes that nodes' connectivity is well-explained by their degrees, which makes modularity a useful objective when the graph is structurally constructed from identity (co-authorship networks, protein-protein interaction networks) but a misaligned objective when the graph is constructed from spatial proximity (such as the kNN keypoint graph of Section 3.3.2).

DMoN [61] converts Equation (3.14) into a differentiable objective by replacing the discrete indicator matrix with a soft assignment $\mathbf{S} \in [0,1]^{N\times K}$ produced by a GNN encoder followed by a per-node softmax over K clusters, and adding two regularisers that prevent degenerate solutions. The full DMoN loss is

$$\mathcal{L}_{\text{DMoN}} = -\frac{1}{2m}\text{Tr}\big(\mathbf{S}^\top \mathbf{B}\mathbf{S}\big) + \lambda_{\text{ortho}} \left\| \frac{\mathbf{S}^\top\mathbf{S}}{\|\mathbf{S}^\top\mathbf{S}\|_F} - \frac{\mathbf{I}}{\sqrt{K}} \right\|_F^2 + \lambda_{\text{cluster}} \|\mathbf{s}_{\text{vol}}\|_1, \tag{3.15}$$

where the first term is the negative modularity (to be minimised), the orthogonality term encourages a balanced partition with cluster-mass approximately uniform, and the collapse-prevention term penalises empty clusters via the cluster-volume vector $\mathbf{s}_{\text{vol}}$. λ_{ortho} and λ_{cluster} are the trade-off coefficients. DMoN was the natural follow-up after the three supervised learned heads of Section 3.2.2–3.2.4 failed to outperform k-means, because it operates on a fundamentally different signal (graph structure) than the embedding-based heads (embedding geometry). Section 3.7 describes the eleven SA-DMoN variants that modify the null model of Equation (3.15) to suit the pose grouping setting, and Chapter 4 reports the resulting evidence that no null-model modification recovers a useful gradient signal on the kNN keypoint graph.

3.3 PROBLEM FORMULATION

This section formalises the multi-person pose grouping problem and fixes the notation used throughout the chapter. The setting is *bottom-up* multi-person pose estimation, in which all keypoints in an image are detected first and then grouped into per-person skeletons, in contrast to *top-down* approaches that first detect a bounding box per person and then run a single-person pose estimator inside each box. The bottom-up family is preferred for crowded scenes where person-detector boxes overlap and split keypoints across boxes, and the grouping stage is the component developed here. Concretely, an off-the-shelf single-image keypoint detector [10] produces a set of two-dimensional keypoint detections, each tagged with one of the seventeen COCO joint types [1], and the grouping stage must partition this set into per-person skeletons without seeing the source image. The decoupling of detection from grouping is the central architectural choice that distinguishes this approach from end-to-end methods such as DEKR [16] or ED-Pose [24]; its consequences are addressed in Section 3.12.

3.3.1 Inputs and outputs

Let an image contain K people. The detector emits N keypoint detections collected into a set $\mathcal{X} = \{(\mathbf{p}_i, t_i, v_i)\}_{i=1}^{N}$, where $\mathbf{p}_i \in [0,1]^2$ is the normalised image-plane position of detection i, $t_i \in \mathcal{T} = \{1,\dots,T\}$ is the joint type with $T = 17$ for COCO, and $v_i \in \{0,1\}$ is a visibility flag indicating whether the detector emitted a confident keypoint at that location. The grouping task is to produce a labelling $\hat{\mathbf{y}} \in \{1,\dots,K\}^N$ that assigns each detection to one of the K person identities, such that $\hat{\mathbf{y}}_i = \hat{\mathbf{y}}_j$

if and only if detections i and j belong to the same person.

During training, ground-truth labels $\mathbf{y}^\star$ are available; at inference, K itself may also be unknown and must be estimated (Section 3.12.3). The headline evaluations assume K is provided by an oracle, which isolates the embedding-quality contribution from the K-estimation error mode; the autonomous pipeline that estimates K at inference is described in Section 3.12 and evaluated separately.

3.3.2 The skeleton graph

The grouping problem is cast as node classification on a graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ constructed from the keypoint detections. Each node $i \in \mathcal{V}$ corresponds to one detection in $\mathcal{X}$, so $|\mathcal{V}| = N$. Each detection contributes a continuous feature vector $\mathbf{x}_i$ whose exact composition depends on whether the depth channel is retained; Section 3.4.3 fixes both variants. The layer-zero input to the embedding network is the concatenation of $\mathbf{x}_i$ with a learnable embedding of the joint type,

$$\mathbf{h}_i^{(0)} = \left[\mathbf{x}_i \,\middle\|\, \mathrm{emb}_{\mathcal{T}}(t_i)\right] \in \mathbb{R}^{d_0}, \tag{3.16}$$

where $\mathrm{emb}_{\mathcal{T}} : \{1, \ldots, T\} \to \mathbb{R}^{32}$ is a learnable lookup table over the seventeen COCO joint types. Keeping the type embedding separate from the spatial features allows the network to learn that a nose differs from a left knee independently of where each one appears in the image. Edges are formed by a k-nearest-neighbour rule in image-plane distance with $k = 8$, chosen to give every node a connected neighbourhood without densifying the graph to the point that attention has no discriminative work to do. The kNN-rule edge set is symmetric and contains no self-loops.

This graph construction has two properties that the methodology below depends on. First, the graph encodes *spatial* proximity, not identity: two nearby left-shoulder detections from different people produce a strong edge, even though they must end up in different communities. The embedding stage must therefore separate them. Second, the joint type t_i is observed at every node, so the architecture can use type-aware structural priors without breaking the unsupervised-at-inference contract. The PG-GAT modifications of Section 3.8 are exactly the mechanisms that exploit this observed-type information.

Figure 3.1. The COCO 17-joint skeleton (left) and a representative kNN graph with $k = 8$ on a two-person scene (right). Nodes are joint detections; skeleton edges (left) encode anatomical adjacency, while kNN edges (right) encode image-plane spatial proximity. A same-type cross-person edge — for example between two left shoulders — is the failure case that the embedding stage of Section 3.5 must resolve.

3.3.3 The three sub-problems

Pose grouping decomposes into three sub-problems that this chapter addresses in turn.

3.3.3.0.1 EMBEDDING. Produce per-node representations $\mathbf{z}_i \in \mathbb{R}^d$ such that the Euclidean structure of the embedding space reflects person identity: joints from the same person are close, joints from different people are far. A contrastive loss with ground-truth person labels is used at training time to push and pull pairs of embeddings according to whether they share a person identity. The embedding network is the dominant axis of variation in the experiments that follow; every PG-GAT, SA-DMoN, hyperbolic, and TriGAT variant in this chapter is a modification of the embedding network alone.

3.3.3.0.2 GROUPING. Given the embeddings $\{\mathbf{z}_i\}$ and the number of people K, recover the partition $\hat{\mathbf{y}}$. A single grouping method is reported as the headline read-out: k-means with $k = K$ on the L2-normalised embeddings, denoted kNN throughout. This restriction is deliberate. Pose grouping is framed here as an unconstrained clustering problem in which the embedding does the discriminative work and the grouping head reads off the answer. Type-constrained, structured-assignment, and other learned grouping methods are treated as cross-paper context and reported only briefly where direct comparison is useful; their detailed investigation belongs to a companion paper outside the present scope.

3.3.3.0.3 K ESTIMATION. Predict K from the embeddings alone, so that the full pipeline can run without an oracle. A small MLP head trained on the embeddings is described in Section 3.12.3. Joint

training of the K head with the embedding network is shown in Chapter 4 to corrupt the embedding; the head is therefore trained on a frozen embedding network.

3.3.4 Evaluation metric

The primary metric is Pose Grouping Accuracy (PGA), the per-joint accuracy of the grouping after Hungarian matching of predicted cluster identifiers to ground-truth person identifiers. The Hungarian algorithm [62] solves the optimal-assignment problem in polynomial time: given a non-negative cost matrix between two equally-sized sets, it returns the bijection that minimises the total cost. PGA uses it to find the relabelling of predicted clusters that maximises agreement with the ground truth, which is necessary because k-means produces arbitrary cluster identifiers that carry no semantic ordering. Concretely, given a predicted labelling $\hat{\mathbf{y}}$ and ground truth $\mathbf{y}^\star$, the optimal cluster-to-person bijection $\pi^\star : \{1,\dots,K\} \to \{1,\dots,K\}$ is computed by Hungarian assignment [62] on the overlap matrix, and PGA is the fraction of joints whose remapped prediction matches the ground truth:

$$\mathrm{PGA}(\hat{\mathbf{y}}, \mathbf{y}^\star) = \frac{1}{N} \sum_{i=1}^{N} \mathbb{1}[\pi^\star(\hat{\mathbf{y}}_i) = \mathbf{y}_i^\star]\,. \tag{3.17}$$

Hungarian matching is necessary because the absolute cluster identifiers produced by k-means (and by any other clustering method) are arbitrary. The same metric is used on synthetic and COCO data; the only thing that changes between settings is the data distribution. A detailed account of the metric, the dataset compositions used for each evaluation row, and the W&B sweep protocol that defends every hyperparameter choice is given in Section 3.13.

3.4 DATASET PIPELINE

Every method in this chapter is trained on synthetic data with ground-truth person labels and evaluated on both synthetic and COCO data. This section describes the synthetic generator, the COCO adapter, the unified preprocessing pipeline that places both sources behind a common interface, and the methodological decision to remove the depth channel before any of the headline experiments were run.

3.4.1 Synthetic generator

The synthetic data is rendered with Blender [63]. A configurable number of mannequin meshes is placed in a 3D scene with procedurally varied camera positions, viewing angles, and lighting. Each frame yields a colour image and a per-person keypoint annotation containing seventeen joint locations in the COCO keypoint format, each joint stored as $[x, y, z, v]$ where (x, y) are pixel-plane coordinates, z is the world-space Euclidean distance from the camera to the joint in metres, and $v \in \{0, 1, 2\}$ is the visibility flag. The visibility flag is set by a Blender raycast against scene geometry, so joints

occluded by other people or scene objects are correctly marked as $v = 1$ rather than dropped to $v = 0$. Out-of-frame joints use the sentinel position $(-1, -1)$ to distinguish them unambiguously from the legitimate top-left corner coordinate $(0, 0)$.

The generated dataset contains 400 training, 100 validation, and 100 test scenes. The person count per scene is sampled uniformly from $\{1, 2, 3, 4, 5\}$ so that the K-estimation head sees the full range it must predict at inference; this distribution also matches the modal range of multi-person COCO images. Filenames follow the convention `<guid>_<n>.png`/`<guid>_<n>.json` to allow filtering by person count without parsing the JSON, and `num_people` is duplicated into the image metadata for dataset statistics.

Figure 3.2. Representative scenes from the synthetic dataset at person counts $K \in \{2, 3, 4, 5\}$. Each panel shows the rendered Blender frame with the 17 per-person ground-truth keypoints overlaid and colour-coded by person identity. The synthetic generator controls camera pose, lighting, person placement, and visibility, and emits exact metric depth at each joint via raycast, which the depth-removal analysis of Section 3.4.4 subsequently discards.

3.4.2 COCO adapter

For evaluation on real images, the COCO 2017 person-keypoint annotations [1] are loaded via the pycocotools API and filtered to images containing at least one person. The native COCO visibility flags ($0 =$ not labelled, $1 =$ occluded, $2 =$ visible) are mapped to the same three-value semantics as the synthetic generator. COCO does not provide a depth channel, so the adapter estimates one via MiDaS [64], a pre-trained monocular depth estimator that produces a per-pixel relative depth map from a single RGB image. The DPT-Hybrid variant is used because it combines convolutional and transformer backbones and is the strongest publicly available checkpoint at moderate inference cost; the model is instantiated once at adapter construction time and the depth value at each labelled keypoint

pixel is sampled into the z slot of the keypoint vector. This produces a $[x, y, z, v]$ representation that is dimensionally identical to the synthetic format and that the downstream pipeline cannot distinguish from synthetic data.

The decision to estimate depth via MiDaS rather than to drop the depth channel for COCO was made to keep the network architecture identical across data sources. Section 3.4.4 below describes the subsequent measurement that led to depth being removed for every method in this chapter.

3.4.3 Unified pipeline

Both adapters emit the same per-image record: the image tensor at $[3, H, W]$, a list of per-person keypoint tensors at $[17, 4]$, an image identifier, and the person count. A single `PoseDataset` sits behind both adapters and applies a common transform: each image is resized to fit within `target_size` $\times$ `target_size` preserving aspect ratio, symmetrically padded to the exact target dimension, and the keypoint coordinates are scaled and shifted to match; the z and v columns are passed through unchanged and joints with $v = 0$ are not transformed at all. A custom collate function in the dataloader stacks the image tensors but keeps the per-image keypoint lists as a list-of-lists, because the per-image person count is variable and PyTorch's default collate cannot batch variable-length structures.

The preprocessor converts each batch into PyTorch Geometric graphs [65], one graph per image. Joints with $v = 0$ are excluded entirely from the graph rather than represented as zero-feature nodes, so that the message passing has no spurious nodes to aggregate over. Each surviving joint becomes a node with raw feature vector

$$\mathbf{x}_i = [x_{\text{norm}},\ y_{\text{norm}},\ z_{\text{norm}},\ v_{\text{norm}}] \in \mathbb{R}^4, \tag{3.18}$$

where the spatial coordinates are normalised to $[0, 1]$ by the image dimensions, z_{norm} is a per-graph min-max normalisation of the depth over valid joints, and v_{norm} maps the three visibility levels to $\{0.0, 0.5, 1.0\}$. The per-graph depth normalisation is what makes MiDaS-estimated COCO depth and Blender-rendered synthetic depth dimensionally compatible: both sources land in $[0, 1]$ regardless of absolute scale. The vector $\mathbf{x}_i$ is the raw *preprocessed* feature used by the rest of the chapter; the GAT layer-zero input $\mathbf{h}_i^{(0)}$ of Equation (3.16) concatenates $\mathbf{x}_i$ with the joint-type embedding. Separate tensors per graph hold the joint-type indices (consumed by the type embedding) and the ground-truth person labels (consumed by the contrastive loss at training time). Edges are built by k-nearest neighbours in $(x_{\text{norm}}, y_{\text{norm}}, z_{\text{norm}})$ space with $k = 8$, chosen as described in Section 3.3.2.

3.4.4 Removing the depth channel

The depth channel was removed from every method reported in the headline evaluations, on the basis of a single diagnostic experiment whose result is reported in Chapter 4. The methodological motivation is recorded here so that the rest of the chapter can refer back to it.

With depth, kNN clustering on contrastive GAT embeddings reaches a synthetic-test PGA of 0.994; the problem is effectively solved by spatial proximity in three dimensions. The metric depth produced by the Blender renderer is exact ground truth, so two people standing at different camera distances are separated by the z coordinate alone, and the GAT has very little discriminative work to do. This artificially inflated all synthetic baselines and left no room for the learned heads or for SA-DMoN to demonstrate added value: any small improvement on a near-perfect baseline is statistically indistinguishable from noise.

On COCO, the depth signal comes from MiDaS rather than from ground truth. MiDaS produces a *relative* depth ordering with no metric scale, and the per-image scale varies with the image content. The synthetic-to-COCO domain gap is therefore amplified by the z channel: a synthetic model trained on metric depth has learned to read z at a scale that does not exist in the MiDaS output. Removing depth removes this domain-gap component and forces every method in this chapter to operate on the same two-dimensional signal that a real bottom-up pipeline has access to without an external depth estimator.

The removal is implemented via a `use_depth` flag in the preprocessor configuration. When the flag is off, the raw feature vector of Equation (3.18) reduces to

$$\mathbf{x}_i = [x_{\text{norm}},\, y_{\text{norm}},\, v_{\text{norm}}] \in \mathbb{R}^3, \tag{3.19}$$

the kNN graph is built on $(x_{\text{norm}}, y_{\text{norm}})$ only, and the input dimension of the GAT first layer is reduced accordingly via Equation (3.16). Every result reported in Chapter 4 from the baseline kNN onward uses this no-depth configuration, unless the row is explicitly labelled as a depth-on diagnostic.

3.5 BASELINE: STANDARD GAT WITH CONTRASTIVE LOSS

The baseline against which every subsequent method is compared is a standard graph attention network trained with a contrastive loss on the synthetic dataset and read out with k-means clustering on the L2-normalised embeddings. This baseline establishes the bar that any modification of the embedding

network must clear, and it fixes the training schedule, optimiser, and clustering protocol that all subsequent methods inherit.

3.5.1 Architecture

The network consumes the graph $\mathcal{G}$ constructed in Section 3.3.2 and produces a per-node embedding $\mathbf{z}_i \in \mathbb{R}^d$ with $d = 128$. The layer-zero GAT input $\mathbf{h}_i^{(0)}$ is the concatenation defined in Equation (3.16): the raw preprocessed feature $\mathbf{x}_i \in \mathbb{R}^3$ of Equation (3.19) together with a learnable joint-type embedding of dimension 32, giving a layer-zero representation of dimension $d_0 = 35$ in the no-depth configuration that the baseline uses by default ($d_0 = 36$ in the depth-on diagnostic variant of Equation (3.18)).

Two GATv2 [48] layers do message passing over $\mathcal{G}$, each layer using four attention heads with hidden dimension 64. The choice of GATv2 over the original GAT [47] is methodologically deliberate: the original GAT computes its attention coefficients with a static linear transformation that is monotonic in the query node's representation, which bounds the expressive power of the attention mechanism; GATv2 swaps the order of the linear projection and the LeakyReLU to recover a fully dynamic attention function, and prior work [48] shows that this matters whenever the same query must attend differently to neighbours of different types. The pose-grouping task is exactly such a setting, because a single keypoint must attend differently to a same-type neighbour than to a skeleton-adjacent neighbour of a different type. GATv2 is therefore the attention primitive used throughout; the relevant message-passing definitions are recalled in Section 3.1.3.

A linear projection maps the post-message-passing representation to the embedding dimension, and an L2 normalisation step places every embedding on the unit hypersphere $\mathbb{S}^{d-1}$. The hypersphere geometry is the working assumption of the contrastive loss below; it also gives the downstream k-means a bounded space to operate in. The complete architecture is summarised in Table 3.1.

Component	Setting
Attention primitive	GATv2 [48]
Number of GAT layers	2
Number of attention heads	4
Hidden dimension	64
Output (embedding) dimension	128
Joint-type embedding dimension	32
Layer normalisation	yes
Output normalisation	L2 (unit hypersphere)
Dropout	0.1

Table 3.1. Baseline GAT architecture used as the comparison point for every method in this chapter. The same hyperparameters are inherited by every method below that does not explicitly sweep over them; the architecture sweep that searches over these axes for the PG-GAT centrepiece is described in Section 3.8.5.

3.5.2 Contrastive loss

The embedding network is trained with a margin-based contrastive loss [66] on cosine similarity. For a graph of N keypoints with ground-truth person labels $\mathbf{y}^\star \in \{1,\dots,K\}^N$, let $\cos(\mathbf{z}_i,\mathbf{z}_j) = \mathbf{z}_i^\top \mathbf{z}_j$ be the cosine similarity between two L2-normalised embeddings. Pairs are partitioned into a positive set $\mathcal{P}^+ = \{(i,j) : \mathbf{y}_i^\star = \mathbf{y}_j^\star, i \neq j\}$ and a negative set $\mathcal{P}^- = \{(i,j) : \mathbf{y}_i^\star \neq \mathbf{y}_j^\star\}$. The loss penalises positive pairs whose similarity is below a margin γ and negative pairs whose similarity is above the symmetric margin $-\gamma$:

$$\mathcal{L}_{\text{contrastive}} = \frac{1}{|\mathcal{P}^+|} \sum_{(i,j)\in\mathcal{P}^+} \max\big(0,\ \gamma - \cos(\mathbf{z}_i,\mathbf{z}_j)\big) \ + \ \frac{1}{|\mathcal{P}^-|} \sum_{(i,j)\in\mathcal{P}^-} \max\big(0,\ \cos(\mathbf{z}_i,\mathbf{z}_j) + \gamma\big). \tag{3.20}$$

The margin γ is fixed at 0.5 across all experiments, a value that the PG-GAT loss sweep of Section 3.8.5 subsequently confirms as near-optimal across a 24-run Bayesian search. Equation (3.20) is differentiable in the embeddings, and the gradient pulls positive pairs together until their similarity exceeds γ and pushes negative pairs apart until their similarity drops below $-\gamma$, saturating outside the active band so that easy pairs do not dominate the gradient signal. The symbol γ for the margin is reserved here to avoid collision with the symbol m used in Section 3.2.5 for the total edge count of a graph.

3.5.3 Training schedule and grouping read-out

The baseline is trained for 50 epochs on the 400-scene synthetic training split with batch size 4, using the AdamW [67] optimiser at learning rate 10^{-3}, a cosine-annealing schedule [68] decaying to 10^{-5},

and weight decay 10^{-4}. Gradient norms are clipped to a maximum of 1.0 to suppress occasional outlier batches dominated by a single high-density scene. Validation runs every five epochs against the 100-scene synthetic validation split using the PGA metric of Equation (3.17). The checkpoint with the best validation PGA is retained as the final model.

At inference, the trained embedding network is run once per image to produce $\{\mathbf{z}_i\}_{i=1}^{N}$, and the partition is obtained by k-means with $k = K$ on the L2-normalised embeddings using the scikit-learn [69] implementation with ten random initialisations and a fixed seed. PGA is then computed by Equation (3.17) after Hungarian matching of the resulting cluster identifiers to the ground-truth person identifiers.

Two baseline configurations are reported in Chapter 4: the depth-on variant, which uses the four-dimensional node feature of Equation (3.18) and is included as a diagnostic of the synthetic-easy regime described in Section 3.4.4; and the no-depth variant, which uses Equation (3.19) and is the baseline against which every later method is compared. All methods from Section 3.6 onward inherit the no-depth configuration, the 50-epoch schedule, and the optimiser settings of this section unless explicitly overridden. The architecture hyperparameters of Table 3.1 are themselves the hand-picked baseline values that the learned-heads and SA-DMoN sections inherit; PG-GAT later replaces them with a Bayesian-sweep-selected configuration that grows the network to three layers and an output dimension of 256 (Section 3.8.5). The baseline two-layer configuration is therefore the bar that any modification of the embedding network or the grouping head must clear at matched compute, not the architecture that PG-GAT ultimately uses.

3.6 LEARNED GROUPING HEADS

Three learned grouping heads are constructed on top of the baseline GAT embedding network of Section 3.5. Each is a different attempt to push the grouping decision into a learnable component rather than reading off the structure of the embedding space with k-means. The mathematical preliminaries for each head are given in Section 3.2; this section describes how each was adapted to the pose-grouping problem, the loss it minimises during joint training with the GAT, and the read-out used at inference. None of the three improves on plain k-means in the headline evaluations of Chapter 4, and the resulting evidence motivates the shift of attention to the embedding network itself in Section 3.8.

3.6.1 Deep embedded clustering on pose embeddings

The DEC head [53] treats grouping as a fully unsupervised problem on top of the GAT embeddings. At training time the head maintains K cluster centres $\boldsymbol{\mu}_1, \ldots, \boldsymbol{\mu}_K \in \mathbb{R}^d$ in the same embedding space as the GAT output. At each forward pass over a scene with K ground-truth people, the centres are initialised by k-means++ [51] on the current GAT embeddings of that scene; this per-scene initialisation is the methodological adaptation that makes DEC applicable to a setting where every image has a different K rather than a fixed cluster count over a single dataset.

Given embeddings $\{\mathbf{z}_i\}$ and centres $\{\boldsymbol{\mu}_k\}$, the soft assignment under a Student's t-distribution with one degree of freedom [54] is

$$q_{ik} = \frac{\left(1 + \|\mathbf{z}_i - \boldsymbol{\mu}_k\|_2^2\right)^{-1}}{\sum_{k'} \left(1 + \|\mathbf{z}_i - \boldsymbol{\mu}_{k'}\|_2^2\right)^{-1}}, \tag{3.21}$$

and the sharpened target distribution is

$$p_{ik} = \frac{q_{ik}^2 / f_k}{\sum_{k'} q_{ik'}^2 / f_{k'}}, \quad f_k = \sum_i q_{ik}. \tag{3.22}$$

The loss is the Kullback–Leibler divergence between target and prediction, $\mathcal{L}_{\text{DEC}} = \sum_{i,k} p_{ik} \log(p_{ik}/q_{ik})$, combined additively with the contrastive loss of Equation (3.20). The target $\mathbf{p}$ is computed with `torch.no_grad` and treated as a fixed target so the gradient flows only through $\mathbf{q}$: the head learns to make its soft assignments more confident over time, and the underlying GAT receives gradient signal through the dependence of $\mathbf{q}$ on the embeddings.

DEC is the only head in this section that does not consume ground-truth person labels in its own loss. The contrastive loss still uses the labels, so the GAT receives supervised signal; the DEC head itself is unsupervised and the methodological prediction from this asymmetry — the head can only sharpen assignments that are already roughly correct, it has no signal to fix wrong ones — is corroborated by the result in Section **??**.

3.6.2 Slot attention on pose embeddings

The slot attention head [55] is supervised by the ground-truth person labels via Hungarian-matched cross entropy. K slot vectors $\mathbf{s}_1, \ldots, \mathbf{s}_K \in \mathbb{R}^d$ are initialised independently for each scene by sampling from a learned Gaussian distribution $\mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\sigma}^2\mathbf{I})$ whose parameters are trained jointly with the GAT. The slots then iteratively refine themselves via a competitive attention mechanism over the GAT embeddings. At each iteration t, queries are obtained by layer-normalising the slots, keys and values are obtained by layer-normalising the embeddings, and an attention matrix is computed by softmax

across slots (rather than the standard softmax across keys), which enforces competition between slots for each joint. The matrix is then row-normalised and used to weight the values into per-slot updates, which a GRU cell [56] mixes into the previous slot state, followed by a residual feed-forward block. After $T = 7$ iterations the final slots produce logits $\ell_{ik} = \mathbf{z}_i^\top \mathbf{W}_{\text{key}} \mathbf{s}_k$ for each joint–slot pair.

The training loss is Hungarian-matched cross entropy: the optimal cluster-to-person bijection $\pi^\star$ is computed by Hungarian assignment on the per-slot mean prediction probability against each ground-truth person, and the cross-entropy loss is computed against the permuted ground-truth labels $\pi^\star \circ \mathbf{y}^\star$. At inference the slots are run identically and the argmax of the logits gives the partition. Iteration count and learning-rate schedule are fixed by the values that stabilised the isolation tests recorded earlier in this chapter: seven iterations rather than the standard three, and a cosine schedule decaying from 10^{-3} to 10^{-5} to avoid the overshoot that a constant learning rate produced at three iterations.

3.6.3 Graph partitioning on pose embeddings

The graph partitioning head reframes grouping as binary edge classification on the keypoint graph, rather than as an explicit partition into K clusters. A pair MLP $\phi_{\text{pair}} : \mathbb{R}^{2d} \to \mathbb{R}$ predicts, for every pair of joints (i, j) in the graph, the logit of the event "i and j belong to the same person". The input to the MLP is the concatenation of the embedding difference and the embedding element-wise product:

$$\phi_{\text{pair}}\big([\,\mathbf{z}_i - \mathbf{z}_j \,\big\|\, \mathbf{z}_i \odot \mathbf{z}_j\,]\big) \in \mathbb{R}. \tag{3.23}$$

This is the standard pair-feature construction from face-clustering GNN literature [59]: the difference encodes direction in embedding space and the product encodes co-activation magnitude.

The loss is binary cross entropy over all $\binom{N}{2}$ pairs. Positive pairs (same person) are far less numerous than negative pairs; for the synthetic scenes with five people and seventeen joints, the ratio is approximately 680 positives to 2890 negatives. The positive class is up-weighted by $\texttt{pos\_weight} = \min(10, n_{\text{neg}}/n_{\text{pos}})$ to prevent the loss being dominated by negative pairs. At inference the predicted affinities are thresholded at 0.5 and connected components on the resulting binary affinity graph are recovered by union–find.

Graph partitioning is the only head of the three for which K is not required at inference: the number of connected components is read directly from the predicted affinities, which makes the head structurally appealing for the autonomous-pipeline use case of Section 3.12. The Chapter 4 results explain why

this structural advantage does not translate into a competitive transfer result.

3.6.4 Joint training schedule

All three heads are trained jointly with the GAT for 50 epochs on the synthetic training split using the optimiser settings of Section 3.5.3. The total loss is the sum of the contrastive loss of Equation (3.20) and the head-specific loss with a head weight set to 1.0; this configuration was retained from the initial isolation tests for each head on the basis that none of the heads benefited from a sweep over the head weight in early experiments, and any deeper sweep would have run against the resource budget allocated to the eventually successful PG-GAT investigation. Validation, checkpointing, and the PGA read-out follow the conventions of Section 3.5.3; the head's own forward pass is used at inference for DEC and slot attention, while the partition head's connected components are read directly as the final assignment.

3.7 SA-DMON — SKELETON-AWARE DEEP MODULARITY NETWORKS

The three learned heads of Section 3.6 all failed to outperform plain k-means on embeddings produced by the same GAT backbone. This section describes a different family of attempts: keep the GAT backbone but replace the grouping head with a Deep Modularity Network [61], in which the head's loss is a differentiable relaxation of the Newman modularity [60] of the predicted partition. The vanilla DMoN baseline and its eleven skeleton-aware variants all plateau in a narrow band well below the k-means baseline, and the resulting diagnosis — the modularity null model is structurally incompatible with the spatial-proximity skeleton graph — is the second piece of evidence (after Section 3.6) that the embedding network rather than the grouping head is the binding constraint.

3.7.1 Vanilla DMoN

Given the keypoint graph $\mathcal{G}$ of Section 3.3.2 with adjacency matrix $A \in \{0,1\}^{N\times N}$, DMoN learns a soft cluster-assignment matrix $\mathbf{S} \in [0,1]^{N\times K}$ with row-stochastic constraint $\sum_k S_{ik} = 1$ by a single linear projection of the GAT embedding followed by a softmax. The training loss is a sum of three terms [61]:

$$\mathcal{L}_{\text{DMoN}} = -\frac{1}{2m}\operatorname{Tr}\left(\mathbf{S}^\top \mathbf{B}\mathbf{S}\right) + \lambda_{\text{ortho}} \left\| \mathbf{S}^\top\mathbf{S}/\|\mathbf{S}^\top\mathbf{S}\|_F - \mathbf{I}/\sqrt{K}\right\|_F^2 + \lambda_{\text{cluster}} \|\mathbf{s}_{\text{vol}}\|_1, \tag{3.24}$$

where $\mathbf{B} = \mathbf{A} - \mathbf{d}\mathbf{d}^\top/(2m)$ is the modularity matrix with degrees $\mathbf{d}$ and total edge count $m = \frac{1}{2}\sum_{ij} A_{ij}$, the spectral term maximises modularity, the orthogonality term encourages cluster-balanced assignments, and the collapse-prevention λ_{cluster} term penalises empty clusters. A supervised cross-entropy term against the ground-truth person labels $\mathbf{y}^\star$ is added with weight $\lambda_{\text{type}} = 1$ to keep the head's predicted partition aligned with identity rather than with the modularity argmax.

This vanilla configuration is reported in Chapter 4 as the SA-DMoN starting point. Its failure mode — modularity is maximised when grouping spatially proximal nodes together, but pose grouping requires separating spatially proximal nodes that belong to different people — motivates the skeleton-aware null model of the next subsection.

3.7.2 SA-DMoN v1: a skeleton-aware null model

The v1 variant replaces the degree-based null model $\mathbf{N}_{\text{deg}} = \mathbf{d}\mathbf{d}^{\top}/(2m)$ with a null that combines spatial proximity and skeleton type adjacency. The spatial component is a Gaussian kernel on normalised keypoint positions

$$P_{ij} = \exp\Big(-\frac{\|\mathbf{p}_i - \mathbf{p}_j\|_2^2}{2\sigma^2}\Big), \tag{3.25}$$

where σ is the bandwidth (treated below as a hyperparameter axis). The type component is a fixed 17×17 affinity matrix T derived from breadth-first search on the COCO skeleton graph: directly connected joint-type pairs receive $T_{ab} = 1.0$, two-hop pairs 0.5, three-hop pairs 0.25, and any pair beyond three hops or topologically disconnected receives 0. The combined skeleton-aware null entry between node i of type a and node j of type b is

$$R_{ij} = T_{ab} \cdot P_{ij}, \tag{3.26}$$

which is then normalised to preserve the total edge mass $\tilde{R}_{ij} = R_{ij} \cdot \big(2m/\sum_{ij} R_{ij}\big)$. The modularity matrix becomes $\mathbf{B}_{\text{pose}} = \mathbf{A} - \tilde{\mathbf{R}}$ and is substituted into Equation (3.24) in place of $\mathbf{B}$. Everything else — the assignment head, the orthogonality and collapse terms, the supervised type loss — is unchanged.

The bandwidth σ is the single sensitive hyperparameter of v1, and four parameterisations were tried in turn to identify the right operating point. Each configuration corresponds to one row of the SA-DMoN ablation in Chapter 4.

1. **Learnable, unconstrained.** $\sigma = \exp(\xi)$ with ξ a free parameter initialised at $\log 0.2$. Lets the optimiser choose the bandwidth.
2. **Learnable, clamped.** σ as above, with the bandwidth clamped to $[0.05, 0.5]$ in the forward pass. Prevents the divergence seen in the unconstrained case.
3. **Learnable, clamped, $\lambda_{\text{spectral}} = 1$.** Same as the clamped variant with the spectral term re-weighted to equal the supervised loss. Tests whether the modularity signal is helpful when given more influence.

4. **Fixed at the per-graph median pairwise distance.** Recomputed per scene with `torch.no_grad`, eliminating the optimiser's incentive to manipulate σ. Tests a bandwidth that adapts to scene scale without being directly optimised.
5. **Spectral term removed.** Sets $\lambda_{\text{spectral}} = 0$ while keeping the skeleton-aware null in place for diagnostic logging. Isolates the supervised type loss from the modularity-driven gradient.

These five v1 configurations are the methodological basis of SA-DMoN Experiments 4–8 in Chapter 4.

3.7.3 SA-DMoN v2: decoupled adjacency and entropy type loss

The diagnostic that motivates v2 is that the spatial adjacency $\mathbf{A}$ and the spatial null $\mathbf{P}$ share the same Gaussian-of-distance structure, so the modularity matrix $\mathbf{B}_{\text{pose}} = \mathbf{A} - \tilde{\mathbf{R}}$ partially cancels out near the bandwidth. The v2 variant addresses this by decoupling adjacency from the geometric null and by replacing the ReLU-based supervised type loss with an entropy-based formulation. Two independent fixes that can be combined, giving four configurations.

3.7.3.0.1 FIX 1: FEATURE-BASED ADJACENCY. The adjacency used inside the spectral loss is recomputed from the current GAT embedding rather than from the geometric kNN graph of Section 3.3.2. The feature-graph adjacency is constructed by k-nearest neighbours in the embedding space with $k = 8$; the geometric graph is retained for message passing, so only the modularity-loss adjacency is replaced. The feature graph is regenerated each forward pass as the GAT weights change.

3.7.3.0.2 FIX 2: ENTROPY TYPE LOSS. The v1 supervised term is replaced with a soft entropy $\mathcal{L}_{\text{type}}^{\text{v2}} = -\sum_{i,k} S_{ik} \log p_{ik}^{\text{type}}$ where p_{ik}^{type} is the probability that cluster k contains keypoint type t_i estimated from the current assignment statistics. The entropy formulation produces smoother gradient signal than the v1 threshold-based ReLU, at the cost of removing the discrete hinge that previously gave the loss its sharpness.

3.7.3.0.3 CONFIGURATIONS. Four v2 ablation rows are reported in Chapter 4: feature adjacency alone with $\lambda_{\text{spectral}} = 1.0$, feature adjacency alone with $\lambda_{\text{spectral}} = 0.1$ (matching v1's spectral weight), entropy type loss alone with the original spatial adjacency, and the combined feature-adjacency-plus-entropy configuration.

3.7.4 Training schedule

All eleven SA-DMoN variants (vanilla DMoN, five v1 configurations, four v2 configurations, and a no-spectral diagnostic) are trained jointly with the GAT for between 100 and 150 epochs on the synthetic training split, using the optimiser settings of Section 3.5.3. The 100-epoch schedule was reserved for variants whose head loss was confirmed to have plateaued by 100 epochs, while 150 was used for variants where the head loss was still drifting at 100 epochs to give them every opportunity to escape the plateau. None of the configurations converges to a regime that beats the kNN-on-baseline-embeddings number; the resulting evidence is the subject of Section **??** and the discussion of modularity-objective alignment is the subject of Section 5.6.

3.8 PG-GAT — POSE GROUPING GRAPH ATTENTION NETWORK

This section describes the methodological centrepiece of the work: PG-GAT, a skeleton-aware modification of the baseline GATv2 embedding network of Section 3.5. Three modifications to the attention mechanism are introduced, each motivated by a specific failure mode of the baseline and each independently configurable for ablation, and a two-stage training schedule (synthetic pre-training followed by COCO fine-tuning) that is itself defended by a Bayesian sweep over fine-tune hyperparameters. Every PG-GAT design choice is justified by reference to a sweep, an ablation, or a prior failure recorded earlier in this chapter.

3.8.1 Design intuition

The motivation for PG-GAT follows directly from the experimental evidence presented in Section 3.6 and Section 3.7. Three independent learned grouping heads (DEC, slot attention, graph partitioning) and ten variants of a skeleton-aware modularity objective (SA-DMoN v1 and v2) all failed to outperform plain k-means clustering on the standard GAT embeddings. The persistent failure of learned heads on the same embedding rules out the grouping head as the binding constraint and identifies the embedding network as the dominant axis of variation. PG-GAT acts on the embedding stage alone: the grouping read-out remains the same k-means as in the baseline, and every modification described below changes how the attention mechanism processes the skeleton graph rather than introducing a new learned component on top of the embedding.

The standard GATv2 attention function [48] is type-agnostic with respect to edges: the same parameters compute attention coefficients for a left-shoulder–left-elbow edge, a left-shoulder–right-shoulder edge, and a left-shoulder–right-ankle edge. These edges carry categorically different information for pose grouping. A same-type pair must belong to different people, an edge that connects two skeletally

adjacent joints is the strongest evidence of a same-person grouping, and a far-apart cross-body pair carries little signal. PG-GAT introduces three mechanisms that surface this categorical edge structure to the attention layer without replacing the GATv2Conv primitive that the baseline relies on.

Figure 3.3. The PG-GAT architecture. Input node features are the keypoint position, the visibility flag, and a learnable joint-type embedding. Modification 1 attaches a four-way edge-category one-hot to every edge that enters the GATv2 attention as a learned additive bias (red). Modification 2 concatenates three geometric edge features (L2 distance, skeleton hop distance, same-type indicator) onto that one-hot before the same edge encoder (blue). Modification 3 splits the multi-head attention into a standard sub-layer over the full neighbourhood and a repulsion sub-layer whose attention mask restricts it to same-type edges only (green). Each modification is independently configurable via a boolean flag, which allows the ablation experiments of Chapter 4 to isolate its marginal contribution.

3.8.2 Modification 1: type-pair attention bias

Every edge $(i, j) \in \mathcal{E}$ is categorised into one of four mutually exclusive classes based on the joint-type pair (t_i, t_j):

- SAME-TYPE — $t_i = t_j$, indicating two keypoints that must belong to different people. Two left elbows in the same image cannot share an identity.
- SKELETAL-NEIGHBOUR — (t_i, t_j) is a directly adjacent pair on the COCO skeleton graph (e.g. shoulder–elbow). The strongest cue for same-person grouping: anatomically connected joints from the same person have consistent spatial relationships.
- SAME-LIMB — (t_i, t_j) lies on the same limb chain but is not directly adjacent (e.g. shoulder–wrist). Provides medium-range structural information.
- CROSS-BODY — otherwise. Carries weak or no structural prior.

The category is represented as a one-hot vector $\mathbf{c}_{ij} \in \{0,1\}^4$ and passed through a small edge-encoder MLP ϕ_{edge} to produce an edge feature $\mathbf{e}_{ij}^{\text{cat}} = \phi_{\text{edge}}(\mathbf{c}_{ij}) \in \mathbb{R}^{d_e}$. This edge feature enters the GATv2 attention coefficient via the standard `edge_dim` mechanism of PyG's `GATv2Conv` implementation [65], which extends the attention pre-activation by an additive learned edge bias:

$$\alpha_{ij}^{(l)} \propto \exp\Big(\mathbf{a}^{(l)\top}\text{LeakyReLU}\big(\mathbf{W}^{(l)}[\mathbf{h}_i^{(l)} \,\|\, \mathbf{h}_j^{(l)}] + \mathbf{W}_e^{(l)}\mathbf{e}_{ij}^{\text{cat}}\big)\Big), \tag{3.27}$$

where the learnable parameters $\mathbf{a}^{(l)}$, $\mathbf{W}^{(l)}$, and $\mathbf{W}_e^{(l)}$ are GATv2 layer-l weights. The four edge categories therefore enter the attention as four learnable additive biases that the network can adjust independently without disturbing the underlying GATv2Conv numerics.

The decision to inject category information as an edge feature rather than as a per-category weight tensor was made on the basis of an initial implementation that used custom scatter-based attention with per-category projection matrices. That implementation failed catastrophically (synthetic PGA 0.60) due to numerical instability in the manual softmax over scattered scores; switching to GATv2Conv with edge features recovered stable training and removed the instability without affecting expressive power.

3.8.3 Modification 2: skeleton-relative position encoding

The baseline GAT carries no explicit spatial information in its attention coefficients: positional information is present only as a node feature and is mixed indiscriminately with the type embedding and visibility. The second modification injects three geometric edge features that surface relative-geometry information directly to the attention mechanism:

- the L2 distance between the two endpoints, $\|\mathbf{p}_i - \mathbf{p}_j\|_2$;
- the skeleton hop distance $d_{\text{hop}}(t_i, t_j)$, defined as the shortest path between the joint types t_i and t_j on the COCO skeleton graph, normalised to $[0,1]$;
- a same-type indicator $\mathbb{1}[t_i = t_j] \in \{0,1\}$.

These three scalars are concatenated with the four-dimensional category one-hot of Modification 1 and the resulting seven-dimensional edge vector is passed through the same edge-encoder ϕ_{edge} as before. The skeleton hop distance is the load-bearing component: it tells the attention layer how anatomically related two keypoint types are, independently of how spatially close they happen to be in the image. A shoulder near an elbow (hop $= 1$) is expected and the attention can up-weight the

message; a shoulder near an ankle (hop ≥ 3) is surprising and is likely to be a cross-person collision that should be down-weighted.

3.8.4 Modification 3: same-type repulsion heads

The third modification dedicates a subset of the multi-head attention to the inter-person discrimination task. In a standard GAT layer with H heads, every head attends to every neighbour of the centre node; the gradient signal for distinguishing two same-type keypoints of different people is therefore averaged across H heads that are simultaneously trying to learn skeletal grouping. The repulsion modification splits the layer into a standard GATv2Conv that attends to all neighbours and a smaller repulsion GATv2Conv whose attention mask is restricted to same-type edges only. The outputs of the two sub-layers are concatenated along the head dimension, giving the same total head count as the baseline.

Formally, let the standard sub-layer use H_{std} heads attending to the full neighbourhood $\mathcal{N}(i)$, and the repulsion sub-layer use H_{rep} heads attending to the same-type neighbourhood $\mathcal{N}_{=}(i) = \{j \in \mathcal{N}(i) : t_j = t_i\}$, with $H_{\text{std}} + H_{\text{rep}} = H$. The layer-$l$ output is then

$$\mathbf{h}_i^{(l+1)} = \left[\, \text{GATv2Conv}_{\text{std}}(\mathbf{h}_i^{(l)}, \mathcal{N}(i)) \,\middle\|\, \text{GATv2Conv}_{\text{rep}}(\mathbf{h}_i^{(l)}, \mathcal{N}_{=}(i)) \,\right], \tag{3.28}$$

where $[\cdot\|\cdot]$ denotes concatenation along the head dimension. The repulsion sub-layer never sees skeletal-neighbour, same-limb, or cross-body edges, and the contrastive loss of Equation (3.20) consequently pushes its parameters to produce discriminative features specifically for the case "two left elbows — whose elbows are they?"

The default configuration reserves a single repulsion head out of four ($H_{\text{rep}} = 1$, $H_{\text{std}} = 3$); this choice was validated by the architecture sweep described in Section 3.8.5, which is the same sweep that fixes the depth, hidden dimension, output dimension, and dropout of the PG-GAT layer stack.

3.8.5 Composing the modifications and training

The three modifications are independently configurable via boolean flags on `SAGATConfig`[1] (`use_type_pair_attention`, `use_position_encoding`, `use_repulsion_heads`), which allows each one to be isolated for the ablation experiments reported in Chapter 4. The PG-GAT model used in the headline evaluations enables all three; the ablation table that reports the marginal contribution of each modification is Table 4.4 in Chapter 4.

[1]The implementation class retains the legacy name `SAGATConfig`; the method is referred to as PG-GAT throughout this document.

3.8.5.0.1 ARCHITECTURE DEFENDED BY BAYESIAN SWEEP. The architecture hyperparameters (number of layers, number of heads, hidden dimension, output dimension, joint-type embedding dimension, dropout) are not inherited from the baseline of Section 3.5. Instead, they are selected by a 32-run Bayesian sweep over the six axes, with the search space chosen to span the smallest values used by prior GNN-on-pose work up to the memory ceiling at batch size 4 on the experimental hardware. Selection metric: synthetic validation PGA of Equation (3.17). Hyperband early termination [70] retires weak configurations at the fifth validation iteration (epoch 25) to keep the sweep within the overnight budget while preserving late-converging configurations. The selected configuration — three layers, four heads, hidden dimension 64, output dimension 256, joint-type embedding dimension 64, dropout 0.1 — is the architecture used by every PG-GAT result in Chapter 4 and Chapter 5. A complementary loss sweep over the contrastive margin, repulsion head count, learning rate, and weight decay returned the defaults of Section 3.5.2 as near-optimal, which is itself recorded as evidence that the contrastive bar is already at the right operating point.

3.8.5.0.2 TWO-STAGE TRAINING. Pre-training is conducted on the 400-scene synthetic dataset of Section 3.4.1 with the optimiser settings of Section 3.5.3. Fine-tuning then proceeds on COCO [1] `train2017` for a small number of additional epochs at a reduced learning rate; this second stage adapts the embedding to the COCO image distribution while retaining the per-person identity structure learned on synthetic data. The exact fine-tune schedule (epochs, learning rate, weight decay) is selected by a third Bayesian sweep over a 12-run search space around the legacy inherited recipe of 20 epochs at 10^{-4} learning rate. The selected configuration — 15 epochs at the sweep-chosen learning rate and weight decay — produces the headline numbers reported in Chapter 4 and is logged under the W&B alias `meng-headline`.

3.8.5.0.3 READ-OUT. The grouping read-out remains the same k-means with $k = K$ on the L2-normalised PG-GAT embeddings as in the baseline of Section 3.5.3. PG-GAT changes the embedding; the head stays simple by design.

3.8.6 Variants explored after PG-GAT

PG-GAT as described in this section is the defended embedding network and the source of the headline result reported in Chapter 4. Three independent variants of the PG-GAT design are also reported in Chapter 4, each one a controlled stress-test of a different design choice rather than a competing answer to the grouping problem. Section 3.9 (PG-GAT v2) tests whether augmenting the positional input with pre-trained visual features extracted by a MobileNetV2 backbone [71] adds identity signal that positions alone cannot provide. Section 3.10 (hyperbolic PG-GAT) replaces the Euclidean output

space with a Lorentz-model hyperbolic manifold, on the theoretical motivation that tree-structured data embeds with less distortion in hyperbolic space. Section 3.11 (TriGAT) replaces the per-node primitive from a single joint to a skeleton triplet of three connected joints, on the motivation that the pivot angle of a triplet is a rotation-invariant articulation descriptor that single-joint embeddings do not directly capture. Chapter 4 reports all three as negative or marginal results; their methodologies are recorded here so that the corresponding Chapter 4 numbers and the Chapter 5 discussion are interpretable.

3.9 PG-GAT V2 — VISUAL FEATURE AUGMENTATION

This section describes the visual-feature-augmented variant of PG-GAT. The variant tests the hypothesis that pre-trained image features sampled at each detected keypoint add identity signal that the positional input of Section 3.3.2 cannot provide. The methodology is recorded here so that the Chapter 4 negative result is interpretable; the discussion of why the visual signal does not translate into a useful headline gain belongs to Section **??**.

3.9.1 Motivation and design constraints

The PG-GAT embedding of Section 3.8 consumes only normalised keypoint positions and joint types. The image itself is consumed only by the upstream keypoint detector, and no per-pixel visual context is propagated to the grouping stage. A complementary lightweight feasibility study (`eval_mobilenet_features.py`) sampled MobileNetV2 [71] features at each detected keypoint and showed that the addition of those features to the input of an otherwise unchanged k-means improves COCO grouping by a positive but diminishing amount as the grouping method becomes structurally stronger: six points on plain k-means, three points on type-constrained k-means, and less than one point on Hungarian-matched type-wise assignment. That diminishing trajectory is a methodological warning sign: the strongest grouping methods barely use the visual signal, which already places a ceiling on what a trained variant can extract from it. The methodology below was designed to give a trained PG-GAT v2 every advantage the feasibility study did not, so that the resulting negative result reported in Chapter 4 cannot be attributed to an underpowered architecture.

3.9.2 Visual feature extraction and caching

The feature backbone is MobileNetV2 [71] pre-trained on ImageNet [72], specifically the output of layer 17, which produces a 320-channel feature map at $1/32$ image resolution. The choice of layer is methodological: shallower layers (mid-level textures and edges) were shown by the feasibility study to hurt every grouping method tested, and only the deepest semantic features produced any gain. The backbone is held frozen throughout; no end-to-end joint training of the backbone with PG-GAT v2 is

attempted, which keeps the comparison against PG-GAT controlled for everything except the visual input.

For each COCO image, the backbone is run once and the resulting feature map is bilinearly sampled at every ground-truth keypoint pixel location; the result is a per-keypoint feature vector $\mathbf{u}_i \in \mathbb{R}^{320}$. This sampling is performed once over the COCO `train2017` and `val2017` splits and the per-image cache is written to disk as a PyTorch Geometric `Data` object with the feature attribute attached at the node level. The cached dataset contains 54,564 training samples and 2,258 validation samples, and removes the backbone forward pass from the training loop entirely. Training on cached features takes a fraction of the time that on-the-fly extraction would, and trades memory for the ability to iterate quickly over PG-GAT v2 design choices.

3.9.3 Architecture and input fusion

The PG-GAT v2 network extends the baseline PG-GAT architecture of Section 3.8 with a visual-feature projection block at the input stage. Each cached MobileNet feature vector $\mathbf{u}_i$ is projected through a learned linear layer to dimension 32, followed by a layer normalisation [73] and a GELU activation [74]:

$$\mathbf{u}_i^{\text{proj}} = \text{GELU}\big(\text{LayerNorm}(\mathbf{W}_v \mathbf{u}_i + \mathbf{b}_v)\big), \quad \mathbf{W}_v \in \mathbb{R}^{32\times 320}. \tag{3.29}$$

The projected vector $\mathbf{u}_i^{\text{proj}} \in \mathbb{R}^{32}$ is concatenated with the positional and joint-type features of Equation (3.19) before the first GAT layer. All three skeleton-aware modifications of Section 3.8 are retained unchanged; visual augmentation is therefore strictly additive to the PG-GAT design rather than a replacement for any of its components.

The total parameter count of PG-GAT v2 is 197,280, of which approximately 27,000 are added by the projection block of Equation (3.29) and the small adjustment to the first GAT layer's input dimension. The remaining 170,560 parameters are inherited from PG-GAT. The MobileNet backbone itself contributes a further 2.2×10^6 parameters which are not optimised but do count toward any apples-to-apples comparison against HigherHRNet's 28.6×10^6; Section **??** addresses the model-size accounting in full.

3.9.4 Training schedule and the extended run

PG-GAT v2 is trained from scratch on the cached COCO training features with the standard contrastive loss of Equation (3.20) and no grouping head. Two training schedules are reported in Chapter 4: a 20-epoch run at learning rate 10^{-3} with cosine decay to 10^{-5}, and a 50-epoch extended run that

resumes from the 20-epoch checkpoint at a halved initial learning rate 5×10^{-4} and continues for 30 additional epochs. The extended schedule is included to control against the possibility that a short training run undersells the architecture's capacity to extract value from the visual features; the result reported in Chapter 4 shows that the additional 30 epochs add at most a few thousandths of a PGA point above the 20-epoch number, which establishes that the comparison against PG-GAT is not limited by undertraining.

The optimiser and clipping settings are inherited from Section 3.5.3. Validation runs every two epochs on the cached COCO validation features, and the checkpoint with the best COCO validation PGA is retained as the final model. The decision to validate on COCO rather than on synthetic data is methodological: the synthetic generator does not produce per-keypoint MobileNet features, and constructing them from rendered images would require running the backbone on a domain it was not trained for and would introduce a confounding domain gap. Reporting only COCO validation isolates the architectural question without that confound; a footnote in Section **??** reports the synthetic transfer separately for completeness.

3.10 HYPERBOLIC PG-GAT

This section describes a variant of PG-GAT in which the embedding space is replaced from Euclidean $\mathbb{R}^d$ to a Lorentz-model hyperbolic manifold of curvature -1, denoted $\mathbb{H}^d$. The motivation is the geometry of skeletons: a human skeleton is a kinematic tree rooted at the pelvis, and a multi-person scene is a forest of such trees. Hyperbolic space embeds trees with exponentially less distortion than Euclidean space [75, 76], because the volume of a hyperbolic ball grows exponentially with radius while the volume of a Euclidean ball grows only polynomially. If PG-GAT is implicitly spending attention capacity learning hierarchical structure in $\mathbb{R}^d$, substituting $\mathbb{H}^d$ should encode that hierarchy for free.

3.10.1 Architecture

The internal message-passing layers of PG-GAT remain Euclidean and use exactly the modifications of Section 3.8. Only the final embedding is projected onto the Lorentz hyperboloid

$$\mathbb{H}^d = \left\{ \mathbf{x} \in \mathbb{R}^{d+1} : \langle \mathbf{x}, \mathbf{x} \rangle_{\mathcal{L}} = -1,\, x_0 > 0 \right\}, \tag{3.30}$$

where $\langle \mathbf{x}, \mathbf{y} \rangle_{\mathcal{L}} = -x_0 y_0 + \sum_{k=1}^{d} x_k y_k$ is the Minkowski inner product. This choice is methodologically deliberate. The hybrid Euclidean-internal / hyperbolic-output design tests whether a hyperbolic output metric improves pose grouping without committing to a full hyperbolic attention rewrite, which would require Möbius operations and Fréchet-mean aggregation throughout the layer stack and would

constitute a much larger change. The Lorentz model is preferred to the Poincaré ball because it is more numerically stable at the embedding magnitudes that arise in practice [77]; the implementation uses the `geoopt` library [78].

A learnable tangent-scale parameter $\tau \in \mathbb{R}_{>0}$ multiplies the tangent vector before the exponential map at the origin, so that

$$\mathbf{x}_i^{\mathbb{H}} = \exp\mathrm{map}_0\big(\tau \cdot \mathbf{v}_i\big), \tag{3.31}$$

where $\mathbf{v}_i \in \mathbb{R}^d$ is the post-layer output of the Euclidean PG-GAT stack and $\exp\mathrm{map}_0$ is the standard Lorentz exponential map at the origin. The scalar τ is initialised at $1/\sqrt{d}$, which is the empirically derived value that places initial pairwise geodesic distances in the range $[0.6, 1.0]$ at the dimensions tested and thereby avoids the exploding-distance failure mode that occurs when the post-layer norm is left at its default of approximately $\sqrt{d}$. Without the scale, the initial pairwise distances would be on the order of 20 and the standard contrastive margin $\gamma = 0.5$ would be vacuously satisfied.

3.10.2 Hyperbolic contrastive loss

The Euclidean contrastive loss of Equation (3.20) operates on cosine similarity on the unit hypersphere. The hyperbolic variant replaces this with a margin-based loss on the Lorentz geodesic distance

$$d_{\mathbb{H}}(\mathbf{x}, \mathbf{y}) = \operatorname{arccosh}\big(-\langle \mathbf{x}, \mathbf{y} \rangle_{\mathcal{L}}\big), \tag{3.32}$$

which is non-negative and zero only at coincident points. With the same positive and negative pair sets $\mathcal{P}^+, \mathcal{P}^-$ as Section 3.5.2, the loss is

$$\mathcal{L}_{\mathbb{H}} = \frac{1}{|\mathcal{P}^+|} \sum_{(i,j)\in\mathcal{P}^+} d_{\mathbb{H}}(\mathbf{x}_i^{\mathbb{H}}, \mathbf{x}_j^{\mathbb{H}})^2 + \frac{1}{|\mathcal{P}^-|} \sum_{(i,j)\in\mathcal{P}^-} \max\big(0,\ \gamma_{\mathbb{H}} - d_{\mathbb{H}}(\mathbf{x}_i^{\mathbb{H}}, \mathbf{x}_j^{\mathbb{H}})\big)^2. \tag{3.33}$$

Positive pairs are pulled to zero geodesic distance; negative pairs are pushed beyond a margin $\gamma_{\mathbb{H}}$. The margin is set to $\gamma_{\mathbb{H}} = 2.0$, calibrated to be roughly twice the initial average pairwise distance under the tangent-scale initialisation above. The Euclidean cosine margin $\gamma = 0.5$ of Equation (3.20) would be meaningless on this metric.

A numerical safeguard is necessary because the broadcast tensor operations used to construct the pairwise distance matrix can produce values slightly below 1 at the diagonal due to floating-point error, and arccosh is undefined there. The implementation calls `.contiguous()` after every `.expand()` on the embeddings and applies `torch.nan_to_num` on the distance matrix as a residual clamp.

3.10.3 Grouping read-out

The downstream k-means read-out of Section 3.5.3 expects Euclidean inputs. The Lorentz embeddings are therefore mapped back to the tangent space at the origin via the logarithmic map $\text{logmap}_0 : \mathbb{H}^d \to T_{\mathbf{0}}\mathbb{H}^d \cong \mathbb{R}^{d+1}$, the leading time coordinate is dropped, and the resulting d-dimensional vector is L2-normalised. The tangent space at a point on a curved manifold is the flat Euclidean space that best approximates the manifold locally at that point — intuitively, the plane that "just touches" the curved surface at the origin without crossing it — so distances measured in the tangent space approximate the true geodesic distances on the manifold for embeddings whose magnitude is kept small by the τ rescaling. This approximation is the design compromise that makes the hybrid architecture feasible and that gives up some faithfulness to the manifold geometry in exchange for compatibility with the Euclidean read-out.

3.10.4 Two pre-registered hypotheses and the dimension axis

The hyperbolic variant is evaluated against two hypotheses fixed before any runs were executed: H1, that hyperbolic embedding matches or beats Euclidean at the same output dimension; and H2, that hyperbolic embedding matches Euclidean at a substantially smaller output dimension. H1 isolates a possible quality advantage, H2 isolates a possible efficiency advantage, and the two together span the space of useful outcomes.

Two single-run ablations are recorded in Chapter 4 to test each hypothesis, both at output dimension 128 for H1 and at output dimension 32 for H2 (a four-fold reduction relative to the PG-GAT default). A subsequent 8-run Bayesian sweep over hyperbolic output dimension, number of layers, and hidden dimension provides the broader defensibility evidence reported alongside the single-run ablations. A separate Option B configuration fine-tunes the dim-32 hyperbolic model on COCO to test whether the efficiency advantage at synthetic dimension survives transfer. Every hyperbolic experiment in Chapter 4 uses the architecture, training schedule, and optimiser settings of Section 3.5.3 and Section 3.8.5 unless explicitly stated.

3.11 TRIGAT — TRIPLET-BASED PRIMITIVE

This section describes a variant of PG-GAT in which the per-node primitive is replaced from a single joint to a skeleton triplet of three connected joints. The motivation is articulation: two people standing in similar absolute positions remain distinguishable by their joint angles (shoulder–elbow–wrist, hip–knee–ankle, head–torso), and the pivot-angle of a triplet is rotation-invariant articulation information that single-joint embeddings do not directly capture. This intuition was converted into the experimental

hypothesis that embedding triplets rather than joints would surface articulation as a first-class signal and thereby produce a more discriminative embedding for the hard case of spatially overlapping people. The result reported in Chapter 4 contradicts this hypothesis; this section describes the methodology in enough detail that the negative result is interpretable.

3.11.1 Triplet enumeration and features

The COCO skeleton graph admits exactly 19 canonical triplets, indexed by joint-type triples (t_A, t_B, t_C) where (t_A, t_B) and (t_B, t_C) are both COCO skeleton edges. The pivot is the middle joint B. For each scene, every detected person contributes at most 19 triplets; triplets with a missing or invisible joint are skipped, which leaves the per-person triplet count between roughly 12 and 19 depending on visibility.

A triplet node carries a continuous feature vector

$$\mathbf{f}_{ABC} = \left[\, \mathbf{p}_B,\ \Delta_{AB},\ \Delta_{CB},\ \cos\theta,\ \sin\theta,\ \|\Delta_{AB}\|,\ \|\Delta_{CB}\| \,\right] \in \mathbb{R}^{10}, \tag{3.34}$$

where $\mathbf{p}_B \in [0,1]^2$ is the pivot position, $\Delta_{AB} = \mathbf{p}_A - \mathbf{p}_B$ and $\Delta_{CB} = \mathbf{p}_C - \mathbf{p}_B$ are the two outgoing bones, and θ is the pivot angle $\angle(A,B,C)$ measured by the cosine and sine of its values (which avoids the wrap-around discontinuity of representing the angle directly). A learnable 19-way triplet-type embedding $\mathbf{e}^{\text{tri}}_{(t_A,t_B,t_C)} \in \mathbb{R}^{16}$ is concatenated with $\mathbf{f}_{ABC}$ to form the layer-zero node feature. The triplet-type embedding plays the same role for triplets that the joint-type embedding of Section 3.3.2 plays for joints.

3.11.2 Triplet graph and embedding network

The triplet graph $\mathcal{G}^{\text{tri}} = (\mathcal{V}^{\text{tri}}, \mathcal{E}^{\text{tri}})$ has one node per detected triplet, so $|\mathcal{V}^{\text{tri}}|$ is roughly $19K$ for a K-person scene with full visibility (comparable in scale to the joint graph's $17K$). Edges are formed by k-nearest neighbours in pivot-position space with $k = 8$, matching the convention of the joint graph. The embedding network on top of this graph is a GATv2 stack with the same depth, width, and head count as the PG-GAT default, and the same contrastive loss of Equation (3.20) is applied at the triplet level with the supervisory label set to the pivot joint's person identity. The per-triplet output is a 128-dimensional embedding on the unit hypersphere.

3.11.3 Voting from triplet clusters to joint labels

At inference, k-means with $k = K$ is applied to the triplet embeddings, which yields a per-triplet cluster label. Each joint participates in up to three triplets, each of which has been assigned an arbitrary cluster identifier by k-means, and a deterministic voting rule converts triplet-cluster labels into joint labels.

The vote weights the pivot triplet at twice the weight of the wing triplets on the basis that the pivot articulation is the most distinguishing feature of any triplet. Ties are broken by the lowest-numbered triplet's cluster. A Hungarian matching of cluster identifiers to ground-truth person identifiers follows to compute the final PGA score per Equation (3.17).

The voting step is the methodological soft spot of the TriGAT design, and the Chapter 4 results identify it as the dominant failure mode: a single mis-assigned triplet in a person can move two or three of that person's joints to a wrong cluster, amplifying small triplet-level errors into joint-level mistakes. The discussion in Section 5.5 treats the voting step in detail.

3.11.4 Training schedule

TriGAT is trained for 50 epochs on the synthetic training split with the optimiser settings of Section 3.5.3. The training budget matches PG-GAT exactly so that the comparison reported in Chapter 4 controls for compute. At inference, k-means with $k = K$ is applied to the triplet embeddings and the resulting partition is converted to joint labels by the voting rule of Section 3.11.3. Constrained-clustering read-outs of the same embeddings exist in companion work and are not reported here per the scoping rule of Section 3.3.3.

3.12 INTEGRATION: DETECTION, K ESTIMATION, AND THE END-TO-END PIPELINE

The methodology developed in the previous sections describes how to build a pose-grouping embedding network under the assumption that the keypoint detector and the number of people K are both provided by an oracle. This section closes that loop: it describes the keypoint-detector that produces the input to PG-GAT at evaluation time, the K-estimation head that removes the oracle dependency, and the end-to-end protocol that drives the headline detector-independent number reported in Chapter 4. The integration is modular by construction: PG-GAT operates on already-detected keypoints, so any keypoint detector can be substituted upstream without retraining the embedding network.

3.12.1 Keypoint detection backbone

The detector used in every end-to-end evaluation of Chapter 4 is HigherHRNet-W32 at the 512-pixel input resolution [10, 9]. This is the smallest publicly available HigherHRNet variant and is integrated via the `simple-HigherHRNet` wrapper around the original COCO-trained weights. The choice of HigherHRNet rather than a more recent detector (DEKR [16], ED-Pose [24], or others) is methodologically deliberate. HigherHRNet's associative-embedding (AE) grouping mechanism [15] is the canonical comparison point for any bottom-up pose-grouping method. AE learns a single

scalar "tag" embedding alongside the keypoint heatmap at every pixel, trained with a loss that pulls same-person tags together and pushes different-person tags apart, and groups detections by clustering on tag similarity. The AE tag head is trained end-to-end with the detector on the COCO [1] training distribution, and any modular grouping method that operates on detached keypoints must justify its existence relative to AE's joint-training advantage.

The detector is held frozen across every evaluation: no fine-tuning, no head replacement, no retraining. HigherHRNet emits per-joint-type heatmaps, its internal non-maximum suppression extracts keypoint candidates, and the candidate set passed into PG-GAT is the same set that HigherHRNet's own AE grouping consumes. A Hungarian assignment between detected keypoints and ground-truth keypoints (with a 10-pixel matching threshold) attaches a ground-truth person identifier to each matched detection, so that the per-detection PGA of Equation (3.17) can be computed for both AE and PG-GAT against the same denominator.

3.12.2 Pose Grouping Accuracy on detected keypoints

The end-to-end protocol computes Equation (3.17) on the matched-detection subset. Detections that fail the 10-pixel matching threshold are excluded from both numerator and denominator, so the metric measures only grouping quality, not detection recall. Two grouping methods are compared against the matched detections: HigherHRNet's native AE grouping (the reference upper bound at 0.985), and the PG-GAT embedding from Section 3.8 followed by k-means with K either provided by an oracle or predicted by the K head of Section 3.12.3. The same detections are passed through both grouping methods, so the gap between them isolates grouping quality net of detection effects.

3.12.3 K estimation

The autonomous pipeline must estimate K from the input alone. Three approaches were investigated; the third is the one used in the headline evaluation.

3.12.3.0.1 EIGENGAP. A classical spectral approach computes the Laplacian of the keypoint-affinity graph and counts the eigenvalues below a gap threshold. The implementation builds the affinity via an RBF kernel on the embeddings and selects K at the largest consecutive eigenvalue gap. The contrastive embeddings produce a dominant first gap ("one cluster vs many"), which means the eigengap heuristic systematically underpredicts at $K = 2$ regardless of the true value. This approach is reported as a baseline only.

3.12.3.0.2 JOINT TRAINING. A small MLP head is trained jointly with PG-GAT on synthetic data to predict K from a concatenation of mean-pooled embeddings, max-pooled embeddings, and the keypoint count divided by 17 (the number of COCO joint types). The MLP minimises L1 loss against the ground-truth K, and its gradient flows back into the embedding network. This configuration produces perfect synthetic K estimation but corrupts the embeddings: the mean-pooled feature is forced to encode count rather than identity, and the contrastive separation collapses. The joint-training configuration is reported as a documented failure mode that informs the frozen-backbone design below.

3.12.3.0.3 FROZEN-BACKBONE TRAINING. The final K head decouples the two objectives. A trained PG-GAT checkpoint is loaded and frozen; a fresh K-head MLP is trained on the resulting embeddings for 50 epochs with L1 loss. The K head receives no contrastive gradient and the embedding network receives no K-prediction gradient, so the embeddings retain the separation structure learned in the original PG-GAT training. The frozen-backbone K head is approximately 17,000 parameters, trains to convergence in under a minute on the synthetic training split, and produces the K-estimation accuracy reported in Chapter 4. For the COCO setting the same architecture is trained on COCO embeddings rather than synthetic to address the keypoint-visibility distribution shift between the two domains.

3.12.4 The autonomous pipeline

The complete autonomous pipeline composes the components above into a system that consumes raw COCO images and emits per-person keypoint assignments without any ground-truth dependency: HigherHRNet detects keypoints, PG-GAT embeds them, the frozen-backbone K head predicts the person count, and k-means with the predicted k produces the partition. PGA against ground-truth person identifiers is computed by Equation (3.17) on the matched detections.

The autonomous pipeline is reported as a separate row in Chapter 4's headline table rather than substituted in for the oracle-K rows, so that the headline detector-independence gap of 0.0208 isolates grouping quality from K-estimation error. The autonomous row is the operational version of the same system; the oracle row is the diagnostic that measures the embedding by itself. This separation matches the convention of every published bottom-up pose-grouping paper used as comparison.

Figure 3.4. The autonomous end-to-end pipeline. HigherHRNet detects keypoints and assigns each a joint type; a Hungarian match attaches a ground-truth person identifier to each detection (used only for evaluation). The frozen PG-GAT network embeds the detections, the frozen K-head MLP predicts the number of people from the embeddings, and k-means with $k = \hat{K}$ produces the partition. Every learned component is shown in red, every frozen-but-pretrained component in blue, and every deterministic transform (Hungarian, k-means) in grey. The pipeline emits per-person keypoint assignments without any ground-truth dependency; Chapter 4 reports the resulting PGA against ground-truth person identifiers on the matched detections.

3.13 EVALUATION PROTOCOL AND HYPERPARAMETER DEFENCE

This section fixes the evaluation protocol that every result in Chapter 4 uses and records the Bayesian sweep methodology that is responsible for every hyperparameter cited in the chapter. The protocol is uniform across the synthetic, COCO oracle-keypoint, and COCO end-to-end settings; only the data distribution and the upstream keypoint source change between them. The sweep methodology underwrites the defensibility of every cited number: no PG-GAT hyperparameter is reported without a sweep that searched its operating range, and no comparison row in Chapter 4 cites a number that was not also reproducible from a logged run identifier.

3.13.1 Datasets and splits

Three evaluation settings are reported, all using the Pose Grouping Accuracy of Equation (3.17) as the metric.

3.13.1.0.1 SYNTHETIC VALIDATION. The 100-scene synthetic validation split of Section 3.4.1. Used as the model-selection signal during training: the checkpoint with the highest synthetic-validation PGA is retained and is the one used in every downstream evaluation row. This is also the metric on which the Bayesian sweeps below are scored.

3.13.1.0.2 SYNTHETIC TEST. The 100-scene held-out synthetic test split. Reported alongside the validation number to verify that the validation-selected checkpoint does not overfit to validation.

3.13.1.0.3 COCO VAL WITH GROUND-TRUTH KEYPOINTS. COCO [1] `val2017` person-keypoint annotations are filtered to images containing at least one person with at least three labelled keypoints, which retains 2,258 images of the 5,000 in `val2017`. The retained set includes both single-person and multi-person images. Single-person scenes contribute trivial grouping decisions and inflate the absolute PGA; a secondary diagnostic with `min_people` ≥ 2 (1195 multi-person images) is also reported in Chapter 4 for transparency, with the caveat that the comparable HigherHRNet AE numbers in the literature are reported on the full filtered set. The headline COCO row uses the full filtered set; the multi-person subset is reported as a stricter diagnostic in the same table.

3.13.1.0.4 COCO VAL END-TO-END. The same image set as above, but the keypoint input is produced by HigherHRNet-W32-512 as described in Section 3.12.1 rather than read from the ground-truth annotation file. This is the only evaluation row that exercises the modular detector-independent claim of the present work. The reference upper bound is HigherHRNet's own AE-grouping output on the same detections, which is the strongest published bottom-up grouping number on this distribution.

3.13.2 Run identification and the W&B logging layer

Weights & Biases [79] is an experiment-tracking service that records the configuration, metrics, source code, and output artifacts of every training run to a central web dashboard and exposes them by a permanent run identifier. It serves here as the queryable provenance layer that connects every cited number in Chapter 4 to the exact configuration and checkpoint that produced it. Every run cited in Chapter 4 is logged to the W&B project `multiperson-pose-grouping` and is identified by an 8-character run identifier that doubles as the on-disk output-directory GUID. The same identifier appears in the on-disk `outputs/<name>/<run_id>/` hierarchy, in the W&B dashboard, and in the evidence table referenced from Chapter 4 for reproducibility.

Three properties of the logging layer matter for the defensibility argument. First, every checkpoint that improves the validation metric is uploaded as a W&B artifact under an auto-rotating `latest` alias, with a global `best` alias maintained per artifact by a post-hoc selection script (`promote_best.py`). Promoted aliases such as `champion-arch`, `champion-loss`, `champion-hyperbolic`, and the headline alias `meng-headline` are unique-per-project and identify the canonical version of each result. Second, the run's full configuration is serialised at `init` time and is recoverable from the

dashboard six months later without having to consult the local file system. This is the mechanism by which any later question of the form "did you try *X*" is answered with the run identifier of the relevant configuration rather than with a verbal reconstruction. Third, all runs that ground a number in Chapter 4 carry the tag `dissertation-final` in addition to a run-type tag (`ablation`, `sweep`, or `finetune`), which permits a single dashboard filter to enumerate every cited run.

3.13.3 Bayesian sweep methodology

Each PG-GAT hyperparameter is defended by a sweep that searched its operating range. Four sweeps were conducted and are referenced from Section 3.8.5 and Section 3.10.4: an architecture sweep, a loss/regularisation sweep, a hyperbolic-dimension sweep, and a fine-tune-recipe sweep. The methodology is uniform across all four.

3.13.3.0.1 SEARCH STRATEGY. Bayesian optimisation as implemented by the W&B sweep agent, with Hyperband early termination [70] configured to retire under-performing configurations after a small number of validation iterations rather than running every configuration to completion. Bayesian search produces a more sample-efficient exploration of the parameter space than random or grid search at the budgets used here (12–32 runs per sweep), and Hyperband retires weak configurations early enough to keep total sweep wall-clock within an overnight job on the experimental hardware.

3.13.3.0.2 SELECTION METRIC. Synthetic-validation PGA of Equation (3.17) for the architecture, loss, and hyperbolic sweeps, and synthetic-validation PGA after COCO fine-tuning for the fine-tune sweep. The selection metric is the same metric that the trainer uses to decide whether to save a new best checkpoint, so the sweep agent's notion of "best" and the headline's notion of "best" coincide. The COCO and end-to-end numbers that appear in the headline row are produced by a separate evaluation script run on the sweep-winner checkpoint after the sweep finishes; selecting on COCO val directly would conflate the architecture choice with the COCO fine-tune effect, which is exactly the confound that the four-sweep structure is designed to eliminate.

3.13.3.0.3 SEARCH-SPACE SIZING. The search-space bounds for each axis are recorded in the sweep YAML configuration and are reported alongside the sweep result in Chapter 4. Bounds are set to span the smallest value used by prior GNN-on-pose work up to the largest value that fits in the memory budget at batch size 4. Any choice outside these bounds is therefore methodologically excluded by the sweep design, not silently dropped.

3.13.3.0.4 POST-SWEEP EVALUATION. Each sweep terminates with the winning configuration promoted to a `champion-<sweep>` alias and an additional evaluation pass that computes COCO oracle-keypoint PGA and COCO end-to-end PGA on the champion checkpoint. These two numbers

populate the corresponding rows of the Chapter 4 headline table. The chain of provenance is: sweep agent run identifier → artifact version → `champion` alias → Chapter 4 number, all visible in the W&B dashboard and the local file system.

3.13.4 The headline provenance

The headline PG-GAT result is the output of a fine-tune sweep over a PG-GAT model whose architecture is itself the winner of an architecture sweep. The provenance is therefore two sweeps deep: the architecture sweep selects the network depth, head count, hidden dimension, output dimension, joint-type embedding dimension, and dropout; the fine-tune sweep then runs over the COCO fine-tune epochs, learning rate, and weight decay with the architecture pinned to the architecture-sweep winner. The loss sweep is reported alongside as evidence that the contrastive margin and repulsion-head count are at the right operating point, and the hyperbolic-dimension sweep is reported alongside as the corresponding evidence for the hyperbolic variant. The W&B alias `meng-headline` identifies the artifact version of the fine-tune-sweep winner and the run identifier `b5noyg7t` identifies the source run. Every number in the headline row of Section 4.6.2 traces to this provenance chain, which is the strongest defensibility claim of the present work: no hyperparameter was hand-picked, and the headline result is reproducible from a logged identifier.

CHAPTER 4 EXPERIMENTAL RESULTS

This chapter reports experimental results for every method presented in Chapter 3, in the order of the investigation. Each section follows the lab-book pattern (setup, table of numbers, interpretation). Failed approaches (learned heads, SA-DMoN, TriGAT, visual-feature augmentation) are reported with their numbers *and* explanations of why they failed. Honest caveats are flagged where the numbers are close calls.

4.1 BASELINE: STANDARD GAT

This section reports the headline numbers for the standard GAT baseline of Section 3.5 on synthetic and COCO data, with and without the depth channel. Two diagnostic measurements drive every downstream methodological decision in this chapter: the depth-on synthetic number that is too high to be informative, and the depth-removal lift on COCO that motivates the no-depth configuration adopted by every subsequent method.

4.1.1 Setup

Two training configurations are reported. Both use the architecture of Table 3.1: two GATv2 layers, four attention heads, hidden dimension 64, output dimension 128, joint-type embedding dimension 32, L2-normalised output on the unit hypersphere. Training follows the schedule of Section 3.5.3: 50 epochs on the 400-scene synthetic training split, AdamW at learning rate 10^{-3} decaying to 10^{-5} under cosine annealing, weight decay 10^{-4}, batch size 4, gradient-norm clipping at 1.0. The only difference between the two configurations is the depth channel. The depth-on variant uses the four-dimensional raw feature of Equation (3.18); the no-depth variant uses the three-dimensional raw feature of Equation (3.19). Both are evaluated on the 100-scene synthetic validation split during training (the model-selection signal) and on the 2258-image filtered COCO val2017 set after training (the zero-shot transfer measurement). The COCO evaluation uses ground-truth keypoint annotations rather than detector output; the detector-based end-to-end evaluation is reported separately in Section 4.9.

4.1.2 Headline numbers

Configuration	Synth val PGA	COCO val PGA (kNN, GT kp)	W&B run
Standard GAT, depth on	0.9990	[Phase C]	`su8rmvti`
Standard GAT, depth off	**0.8783**	**0.8841**	`jkd1ln4a`

Table 4.1. Baseline GAT under both depth configurations. The depth-on row is reported as a diagnostic of the synthetic-easy regime; the depth-off row is the baseline that every subsequent method in this chapter is compared against. Both numbers are reproducible from their W&B run identifiers in the `multiperson-pose-grouping` project.

4.1.3 Interpretation

Three observations follow from Table 4.1.

4.1.3.0.1 1. THE DEPTH-ON SYNTHETIC NUMBER IS UNINFORMATIVE. A synthetic-validation PGA of 0.9990 means roughly 1–2 of every 100 joint assignments is wrong. At that operating point, the dominant axis of variation is per-image stochasticity in k-means initialisation rather than embedding quality. Any modification of the embedding network or the grouping head that lifts this number is statistically indistinguishable from noise, which makes the depth-on configuration a poor measurement bed for the rest of the chapter. The depth-on number is reported here so that the depth-removal decision below is interpretable; no subsequent row in this chapter retains it.

4.1.3.0.2 2. THE DEPTH-ON TRANSFER TO COCO IS THE FAILURE MODE. The depth-on configuration trains on metric depth produced by the Blender renderer and is evaluated on relative depth produced by MiDaS [64] at COCO inference time (Section 3.4.2). The two depth sources differ in scale, sign convention, and noise floor, so the network has effectively learned to read a z coordinate at a scale that does not exist in the COCO input. The resulting COCO transfer gap exceeds 15 percentage points. The methodological response is to remove the depth channel for every subsequent method (Section 3.4.4), which is what the depth-off row of Table 4.1 measures.

4.1.3.0.3 3. THE NO-DEPTH COCO NUMBER IS THE BAR. Standard GAT with no depth scores 0.8841 COCO val PGA (kNN, ground-truth keypoints) on the 2258-image filtered COCO val2017 set. This row anchors the rest of the chapter: every modification of the embedding network in Sections 4.2–4.4 is compared against this baseline at matched compute, and the PG-GAT centrepiece of Section 4.4 lifts the same metric by a margin defended by the architecture sweep of Section 4.5.

The synth val drop from 0.994 to 0.879 in the no-depth configuration is the cost of removing the depth channel on synthetic data; the COCO lift from 0.838 to 0.879 at the same time is the benefit on real data. The cost is paid in a regime that was uninformative anyway, and the benefit is paid in the regime that drives the headline number. The trade is therefore one-sided in favour of the no-depth configuration.

4.2 FAILED LEARNED GROUPING HEADS

This section reports the results for the three learned grouping heads of Section 3.6: DEC, slot attention, and graph partitioning. Each head was trained jointly with the GAT backbone of Section 3.5 under matched optimiser and schedule settings, and evaluated against the no-depth baseline of Section 4.1. None of the three improves on plain k-means on the same embeddings. The collective failure is the first piece of evidence that the binding constraint of the pose-grouping pipeline is the embedding, not the grouping head; the resulting methodological pivot is the SA-DMoN exploration of Section 4.3 and the PG-GAT centrepiece of Section 4.4.

4.2.1 Setup

All three heads share the optimiser, schedule, and node features of the baseline configuration (Section 3.5.3), operate on the no-depth feature vector of Equation (3.19), and are trained jointly with the GAT for 50 epochs. The head-loss weight is fixed at 1.0 for each head, retained from the initial isolation tests recorded earlier in the investigation; early sweeps over the head-loss coefficient produced no configuration that beat the baseline, and the resource budget for deeper sweeps was reallocated to the PG-GAT investigation that subsequently produced the headline result.

Head-specific hyperparameters that warrant explicit justification:

- **DEC.** The Student's t distribution uses one degree of freedom ($\alpha = 1$), the value used by the original DEC paper [53] and validated as numerically stable on the embedding scale of Section 3.5. The target distribution refresh interval is 100 steps, which is the smallest value that empirically prevents the cluster-collapse failure mode observed during isolation tests at single-step refresh [53]. Cluster centres are re-initialised per scene with k-means++ (Section 3.6.1) rather than learned across scenes, because each scene has a different K.
- **Slot attention.** Seven iterations of the competitive attention mechanism, raised from the original-paper default of three on the basis of an isolation-test diagnostic in which three iterations

produced overshoot and slot oscillation late in training. The learning-rate schedule decays cosine from 10^{-3} to 10^{-5}, which stabilises the slot updates at the rate at which they actually converge.

- **Graph partitioning.** Edge-classification threshold 0.5 at inference, which is the standard convention for binary cross-entropy outputs. Class-imbalance handling uses `pos_weight` $= \min(10, n_{\text{neg}}/n_{\text{pos}})$, which up-weights the positive (same-person) class to compensate for the approximate 4 : 1 negative-to-positive ratio in five-person synthetic scenes without letting the weight grow unboundedly in scenes that contain only one or two people.

4.2.2 Per-head results

Head	Synth val PGA	COCO val PGA (kNN, GT kp)	W&B run
Standard GAT (baseline, depth off)	0.8783	0.8841	`jkd1ln4a`
DEC	0.8608	[Phase C]	`5q0p9it9`
Slot attention	0.9025	[Phase C]	`2kq3i8yv`
Graph partitioning	0.9714	[Phase C]	`s3qr04xm`

Table 4.2. Synthetic-validation PGA of the three learned heads against the no-depth baseline. COCO transfer is reported in the columns marked Phase C and is consistent with the inventory measurements (0.788 and 0.752 for slot and partition respectively; DEC was not previously measured on COCO). Bold formatting is reserved for results that exceed the baseline at matched compute; no learned head meets that bar.

4.2.3 Interpretation

Each row of Table 4.2 is symptomatic of a different failure mode, and the three together motivate the pivot to embedding-modification methods that follows.

4.2.3.0.1 DEC: UNSUPERVISED SHARPENING CANNOT FIX WRONG ASSIGNMENTS. DEC scores 0.8608 synth val, 0.0175 below the baseline. The result is consistent with the structural argument of Section 3.6.1: DEC consumes no ground-truth person labels in its own loss and can only sharpen assignments that are already roughly correct. Per-scene k-means++ initialisation produces good initial centres on a well-separated embedding, and the subsequent KL-divergence training adds nothing because the target distribution is constructed from the current soft assignment and the gradient at convergence is near zero. When the initial centres are wrong — which happens whenever two same-type keypoints from different people fall close in embedding space — the sharpening preserves the error rather than correcting it. The same structural limit appears in controlled diagnostics on

fabricated Gaussian-blob embeddings at the joint count of the real problem, where DEC plateaus at approximately 0.76 regardless of the number of training steps.

4.2.3.0.2 SLOT ATTENTION: SUPERVISED BUT CANNOT ENCODE THE TYPE CONSTRAINT. Slot attention is supervised by Hungarian-matched cross-entropy on the ground-truth person labels and scores 0.9025 synth val, 0.0242 above the baseline. This is the strongest of the three learned heads on synthetic data and the only one that beats the no-depth baseline at all on this metric. The improvement does not transfer: inventory measurements record 0.788 COCO PGA at the (depth-on) configuration of the original slot-attention experiments, which is well below the no-depth baseline of 0.879. The structural reason is that slot attention competes over slot identity but does not enforce the type-exclusivity constraint ("two left elbows cannot share a person") that the constrained methods of Section 3.2.1 embed by construction. On the synthetic distribution the embedding-quality lift suffices; on the COCO distribution the embedding noise is larger and the missing type constraint is what fails.

4.2.3.0.3 GRAPH PARTITIONING: CLEANEST SYNTHETIC RESULT, WORST TRANSFER. The graph-partitioning head is the closest of the three to beating the baseline on synthetic (0.9714 vs 0.8783, an improvement of 0.0931) and the worst-transferring of the three on COCO (0.752 inventory, 0.127 below the no-depth baseline). The synthetic gain is real: binary cross-entropy on $\binom{N}{2}$ pairs provides a dense and unambiguous training signal, and connected-components read-out on the thresholded affinity graph never has to commit to a fixed K. The COCO gap is also real: edge-classification accuracy is extremely sensitive to the affinity-construction inputs, and a small shift in the embedding distribution (which is exactly what the synthetic-to-COCO transfer is) is enough to push the binary decisions to the wrong side of the threshold. The partition head therefore overfits the synthetic edge statistics in a way that the other two heads do not.

4.2.3.0.4 CROSS-CUTTING FINDING: THE EMBEDDING IS THE LEVER. The three failure modes are independent — unsupervised sharpening, missing structural prior, and edge-classification fragility — and yet none of the three produces a configuration that transfers a synthetic gain to a COCO gain over the same embeddings. The cross-method evidence is therefore that the grouping head is not the binding constraint on COCO performance: k-means on the same embeddings (the baseline read-out) consistently matches or beats every learned head once the synthetic-to-real shift is in play. This is the methodological pivot point of the chapter. Sections 4.3–4.4 test the contrapositive — that modifications of the embedding network itself, with k-means held fixed, do produce gains that survive transfer.

4.3 SA-DMON

This section reports the results for the eleven SA-DMoN variants of Section 3.7: the vanilla DMoN baseline, five v1 configurations that parameterise the spatial-bandwidth σ of the skeleton-aware null, four v2 configurations that decouple the spectral-loss adjacency from the message-passing adjacency and replace the supervised type loss with an entropy formulation, and a no-spectral diagnostic that removes the modularity term entirely. The collective result motivates abandoning modularity-driven heads as a research direction: every SA-DMoN configuration plateaus inside a 0.77–0.79 band on synthetic validation, well below the no-depth baseline of 0.8783. No bandwidth schedule, no null-model modification, and no type-loss reformulation moves the result above this plateau.

4.3.1 Setup

All eleven configurations share the GAT backbone of Section 3.5 and the optimiser and schedule of Section 3.5.3; the only differences are the head loss and, in v2, the adjacency feeding the spectral term. The v1 configurations train for 150 epochs and the v2 configurations for 100 epochs; the longer v1 schedule was reserved for variants whose head loss had not plateaued by 100 epochs to give every opportunity to escape the plateau, and the v2 schedule was set to the shorter value after empirical verification that all v2 head losses converge well before 100 epochs. The head-loss weight $\lambda_{\text{spectral}}$ is reported per-variant in the table below; the orthogonality and collapse-prevention weights are fixed at 0.1 (the values in the original DMoN paper [61]), and the supervised-CE term retains weight 1.0 throughout.

4.3.2 Headline numbers

ID	Configuration	Synth val PGA	COCO val PGA (kNN, GT kp)	W&B run
	Standard GAT baseline (depth off)	0.8783	0.8841	`jkd1ln4a`
Exp 3-bis	Vanilla DMoN, $\lambda_{\text{spectral}} = 0.1$	0.7879	[Phase C]	`124ykt2c`
Exp 4	SA-DMoN v1, learnable σ, unconstrained	0.7872	[Phase C]	`69yfqshy`
Exp 5	SA-DMoN v1, learnable σ, clamped $[0.05, 0.5]$	0.7918	[Phase C]	`rvd7qaow`
Exp 6	SA-DMoN v1, clamped σ, $\lambda_{\text{spectral}} = 1.0$	0.7892	[Phase C]	`2ok6u2k3`
Exp 7	SA-DMoN v1, σ = per-graph median pairwise distance	0.7769	[Phase C]	`usjwc380`
Exp 8	SA-DMoN v1, spectral term removed	0.7889	[Phase C]	`2ucbx3e0`
Exp v2a	SA-DMoN v2, feature adjacency only, $\lambda_{\text{spectral}} = 1.0$	0.7849	[Phase C]	`x6v3wfnv`
Exp v2b	SA-DMoN v2, feature adjacency only, $\lambda_{\text{spectral}} = 0.1$	0.7925	[Phase C]	`qn7rv98j`
Exp v2c	SA-DMoN v2, entropy type loss only	0.7934	[Phase C]	`zz2sj7an`
Exp v2d	SA-DMoN v2, feature adjacency + entropy type loss	0.7910	[Phase C]	`02jqs33r`

Table 4.3. All eleven SA-DMoN configurations. Synthetic-validation PGA stays within the 0.77–0.79 band across every configuration, spanning a range of 0.0165 across the eleven runs — smaller than the per-image standard deviation of the metric. The variation between rows is statistically indistinguishable from seed noise, which is the strongest possible evidence that the modifications under test do not produce any useful gradient signal on the keypoint *k*NN graph.

4.3.3 Interpretation

The eleven configurations of Table 4.3 are reported as a group rather than as eleven independent ablations, because the dominant signal in the data is the plateau itself rather than any individual row. Three observations follow.

4.3.3.0.1 1. THE PLATEAU IS THE RESULT. Every configuration lands in $[0.7769, 0.7934]$, a range of 0.0165 that is comparable to the per-scene standard deviation of the PGA metric. Whatever the modularity-driven head is doing, it is not converging to a regime that depends on the modifications under test: the spatial bandwidth σ can be learnable, clamped, fixed, or removed entirely; the spectral term can be present or absent; the adjacency can be geometric or feature-based; the type loss can be ReLU-hinge or entropy-based. None of these changes the band by more than 0.02. The plateau height itself is 0.09 below the no-depth baseline, which means the modifications collectively cost embedding quality rather than improving it.

4.3.3.0.2 2. THE BANDWIDTH EXPERIMENTS ISOLATE THE ROOT CAUSE. Experiments 4–7 vary the spatial-bandwidth σ across four parameterisations: learnable-unconstrained, learnable-clamped, learnable-clamped at higher spectral weight, and fixed at per-scene median pairwise distance. The first parameterisation drives σ to extremely large values (the learned σ diverges to approximately 341 under the unconstrained configuration), which collapses the spatial kernel P_{ij} in Equation (3.25) to a near-uniform 1.0, which collapses the skeleton-aware null in Equation (3.26) to the type-affinity matrix alone, which makes the modularity matrix $\mathbf{B}_{\text{pose}} = \mathbf{A} - \tilde{\mathbf{R}}$ nearly equal to the adjacency itself, which gives the spectral loss nothing to do. Clamping σ to $[0.05, 0.5]$ prevents the divergence and the learned value pins at the upper bound, which is the optimiser saying "the loss prefers a weaker null model even with the clamp". Increasing $\lambda_{\text{spectral}}$ to 1.0 amplifies this weakened signal and degrades performance further (-0.0026); the fixed median-distance σ moves to the opposite extreme (null too tight, $\mathbf{B} \approx 0$ on the kNN edges) and degrades performance most of all (-0.0149). The bandwidth has no useful operating point on the keypoint kNN graph.

4.3.3.0.3 3. V2 ISOLATES THE SECOND-ORDER FAILURE MODE AND CONFIRMS IT. The v2 configurations were introduced after the v1 bandwidth diagnosis to test two structural hypotheses: that the spatial-adjacency and spatial-null collinearity could be broken by computing adjacency from features (Fix 1), and that the ReLU-hinge type loss could be smoothed via an entropy formulation (Fix 2). Both fixes were necessary to test; neither produces a configuration that beats v1's best. Feature adjacency at $\lambda_{\text{spectral}} = 1.0$ underperforms because the adjacency now drifts during training and the modularity signal chases a moving target (v2a, 0.7849). Dropping the spectral weight to 0.1 recovers (v2b, 0.7925) but still does not improve on v1's best (0.7918). Entropy type loss alone is neutral (v2c, 0.7934). Combining both fixes is no better than entropy alone (v2d, 0.7910). The diagnosis is therefore complete: modularity is the wrong objective on a spatial-proximity kNN graph, and no null-model or adjacency modification rescues it.

4.3.3.0.4 METHODOLOGICAL PIVOT. The collective evidence of Sections 4.2 and 4.3 is that no modification of the grouping head on top of the standard GAT embedding produces a configuration that beats k-means on the same embedding. This rules out the head as the binding constraint and is the empirical justification for the PG-GAT investigation of Section 4.4, which keeps the head fixed at k-means and modifies the embedding instead.

4.4 PG-GAT — ISOLATION ABLATIONS

This section reports the four-row isolation ablation for the PG-GAT modifications of Section 3.8. Each of the three modifications is enabled in isolation in turn, and the full configuration that enables all three

jointly is reported in the bottom row. Every configuration uses the no-depth baseline architecture of Table 3.1 with only the relevant attention-mechanism modification switched on. The purpose of this ablation is to measure the marginal contribution of each modification at matched training compute, not to defend the absolute scale of the headline number — that defence is reported separately in the architecture sweep of Section 4.5.

4.4.1 Setup

All four configurations share the synthetic-only training regime of Section 3.5.3 and the no-depth feature vector of Equation (3.19). The baseline-architecture layer-stack of Table 3.1 is retained throughout (two GATv2 layers, four heads, hidden dimension 64, output dimension 128, dropout 0.1) so that the ablation rows differ from the baseline only by the attention-mechanism flags and not by depth, width, or training schedule. The three modifications are independently controlled by the boolean flags described in Section 3.8.5: `use_type_pair_attention` (Modification 1), `use_position_encoding` (Modification 2), and `use_repulsion_heads` (Modification 3). The repulsion-heads variant reserves a single head out of four for the same-type sub-layer ($H_{\text{rep}} = 1$, $H_{\text{std}} = 3$). Each configuration is trained for 50 epochs; no COCO fine-tuning is applied here so that the ablation measurement is the synthetic-only embedding quality. The COCO fine-tune effect is reported separately in Section 4.6.

4.4.2 Headline numbers

Configuration	Type-pair	Pos-enc	Repulsion	Synth val PGA	W&B run
Standard GAT (baseline, depth off; COCO 0.8841)	—	—	—	0.8783	`jkd1ln4a`
PG-GAT, type-pair only	on	off	off	0.8825	`8peqkcyh`
PG-GAT, pos-enc only	off	on	off	0.8825	`ffl2tde3`
PG-GAT, repulsion only	off	off	on	0.8834	`gfqkbqnw`
PG-GAT, all three (full; COCO 0.8868**)**	**on**	**on**	**on**	**0.8896**	`r5ekg0zl`

Table 4.4. PG-GAT isolation ablation under matched architecture and matched training compute. The marginal contribution of each modification in isolation is approximately $+0.004$ to $+0.005$ synth val PGA over the baseline, and the full combination is $+0.011$ over the baseline — larger than any single modification and larger than the sum of any two of them measured separately, which is the expected behaviour for modifications that target different failure modes of the underlying attention mechanism.

4.4.3 Interpretation

The ablation establishes two facts that the rest of the chapter relies on.

4.4.3.0.1 1. EACH MODIFICATION IS NON-TRIVIALLY ADDITIVE. The three single-modification rows land within 0.0009 of each other $(0.8825, 0.8825, 0.8834)$, which is well inside seed-noise. The full configuration lifts $+0.0062$ above the strongest single-modification result and $+0.0113$ above the baseline. The full lift is therefore not the sum of the individual lifts but is comparable to it, which is what the design intuition of Section 3.8.1 predicts: the three modifications act on distinct edge structures (type-pair categories, geometric distances, same-type subset) and their respective gradient signals do not destructively interfere when combined.

4.4.3.0.2 2. THE SINGLE-MODIFICATION ROWS ARE NOISY. Reading any single isolation row against the baseline as evidence of a $+0.004$ lift would be honest but underpowered — the per-image PGA standard deviation on the synthetic test set is comparable in scale. The full-configuration row is the load-bearing measurement; the single-modification rows are reported as evidence that no single modification is doing all the work and as the methodological justification for keeping all three in the architecture rather than discarding one.

4.4.3.0.3 3. THE ABSOLUTE NUMBERS ARE BASELINE-ARCHITECTURE NUMBERS, NOT HEADLINE NUMBERS. This ablation uses the two-layer baseline architecture so that the comparison against the baseline is matched. The architecture sweep of Section 4.5 subsequently selects a three-layer, output-dimension-256 configuration that the ablation rows would inherit, which is why the headline result (0.9715 COCO kNN, Section 4.6) is far above the 0.8896 ablation-row maximum reported here. The lift from 0.8896 to the headline is the combined effect of the architecture sweep and the subsequent COCO fine-tune sweep, both of which build on the all-three-modifications configuration whose marginal value over the baseline is what this ablation defends.

4.5 PG-GAT — BAYESIAN ARCHITECTURE AND LOSS SWEEPS

The isolation ablation of Section 4.4 measures the marginal contribution of each PG-GAT modification at the baseline architecture sizing. The contribution of the modifications, however, depends on the architecture that they operate inside: a network with too few layers cannot exploit the additional gradient signal that the modifications inject, and a network that is too wide may overfit the small synthetic training set. This section reports the two Bayesian sweeps that defend the architecture and loss hyperparameters of the PG-GAT centrepiece: the architecture sweep that selects the layer-stack sizing, and the loss sweep that confirms the inherited contrastive-loss hyperparameters are near-optimal.

Both sweeps use the Bayesian optimisation and Hyperband early-termination protocol of Section 3.13.3 and report synthetic-validation PGA as the selection metric.

4.5.1 Architecture sweep

4.5.1.0.1 SETUP. The architecture sweep searches over six axes: number of layers (depth), number of attention heads, hidden dimension, output (embedding) dimension, joint-type embedding dimension, and dropout rate. The search space for each axis is reported in Table 4.5. The lower bound of each axis is set to the smallest value used by prior GNN-on-pose work [41, 17]; the upper bound is set to the largest value that fits in GPU memory at batch size 4 on the experimental hardware. The sweep is configured for 32 runs with Hyperband early termination at the fifth validation iteration (epoch 25), which retires under-performing configurations early while preserving late-converging configurations. The Bayesian search is implemented by the Weights & Biases sweep agent [79] and is logged under sweep identifier `wg95w0k0` in the `multiperson-pose-grouping` project.

Axis	Search space
`num_layers`	$\{2,3,4,5\}$
`num_heads`	$\{2,4,8\}$
`hidden_dim`	$\{32,64,128,256\}$
`output_dim`	$\{16,32,64,128,256\}$
`joint_embedding_dim`	$\{16,32,64\}$
`dropout`	$\{0.0,0.1,0.2,0.3\}$

Table 4.5. Architecture-sweep search space. Each axis spans the smallest value used by prior pose-GNN work up to the memory ceiling at batch size 4.

4.5.1.0.2 RESULT. The winning configuration is reported in Table 4.6. The selected architecture has three layers (one more than the baseline's two), four heads (matching the baseline), hidden dimension 64 (matching the baseline), output dimension 256 (doubled from the baseline's 128), joint-embedding dimension 64 (doubled from the baseline's 32), and dropout 0.1 (matching the baseline). Synthetic-validation PGA of the winning configuration is 0.9634, an improvement of $+0.0738$ over the all-three-modifications ablation row of Table 4.4. Most of the lift comes from the increased output dimension and the third layer, which is consistent with the W&B sweep's parameter-importance analysis: `hidden_dim` is the dominant axis (importance 0.639, correlation -0.80), and `output_dim` is the only positively correlated axis ($+0.50$).

Axis	Selected value
num_layers	3
num_heads	4
hidden_dim	64
output_dim	256
joint_embedding_dim	64
dropout	0.1
Synth val PGA	0.9634
Source run	08d0ffde
W&B alias	champion-arch

Table 4.6. Architecture-sweep winner. This configuration is the architecture used by every PG-GAT result downstream of this section, including the COCO fine-tune sweep of Section 4.6 and the headline number of Section 4.6.2.

4.5.2 Loss sweep

4.5.2.0.1 SETUP. The loss sweep searches over four axes: contrastive margin γ (Equation (3.20)), the number of repulsion heads H_{rep} (Section 3.8.4), learning rate, and weight decay. The architecture is pinned to the champion-arch winner of Table 4.6, so any improvement from this sweep is attributable to the loss hyperparameters rather than to architectural sizing. The search space is centred on the inherited defaults from Section 3.5.3 ($\gamma = 0.5$, $H_{\text{rep}} = 1$, lr $= 10^{-3}$, weight decay $= 10^{-4}$) and extends approximately one order of magnitude in each direction along each numerical axis. The sweep runs 24 configurations with the same Hyperband protocol as the architecture sweep.

4.5.2.0.2 RESULT. The winning configuration (W&B source run mkmtc8om, alias champion-loss) achieves synthetic-validation PGA 0.9617, which is -0.0017 below the champion-arch number. The difference is within the per-image standard deviation of the metric, so the loss-sweep winner is not statistically distinguishable from the architecture-sweep winner. This result confirms that the inherited loss hyperparameters are already near-optimal, which is the methodologically important finding from this sweep: the defensibility argument for $\gamma = 0.5$, $H_{\text{rep}} = 1$, lr $= 10^{-3}$, weight decay $= 10^{-4}$ is now *the value was chosen because no 24-run Bayesian alternative exceeded it*, not *the value was chosen by hand and never re-examined*. The loss-sweep winner is not promoted to the headline checkpoint; the architecture-sweep winner is retained as the source of the architecture used downstream.

4.5.3 Interpretation

The two sweeps reported in this section serve different purposes. The architecture sweep is a search for improvement and produces a configuration whose synthetic-validation PGA is $+0.0738$ above the all-three-modifications ablation row of Table 4.4. The loss sweep is a defensibility exercise and produces a configuration whose synthetic-validation PGA is statistically indistinguishable from the inherited defaults. Both outcomes are useful: the architecture sweep locks in the layer stack that the rest of the chapter inherits, and the loss sweep retires the question "would a different contrastive margin or learning rate change the result." The combined contribution of the two sweeps is that every architecture and loss hyperparameter of the PG-GAT headline is now sweep-defended, and the answer to the question "why this value" for each of the ten hyperparameters in Tables 4.5 and the loss-sweep search space is *a Bayesian sweep over its operating range selected it*.

4.6 COCO FINE-TUNING

This section reports the COCO fine-tune of the architecture-sweep-selected PG-GAT, which produces the headline COCO numbers defended in Chapter 5. Two configurations are reported: the legacy 20-epoch fine-tune recipe inherited from earlier work, and the 12-run Bayesian fine-tune sweep that confirms (and slightly improves on) the legacy recipe. The fine-tune sweep winner is the source of the W&B alias `meng-headline` and the artifact version `pg_gat_finetune_sweep:v20`, and produces the headline COCO numbers cited throughout the rest of the chapter.

4.6.1 Setup

Both configurations start from the `champion-arch` checkpoint of Section 4.5 (architecture-sweep winner, 0.9634 synth val) and fine-tune on COCO [1] `train2017` keypoint annotations. The fine-tune training loop uses the same contrastive loss of Equation (3.20) as the synthetic pre-training, the same AdamW optimiser, and the same cosine learning-rate schedule; only the training-data source changes from the 400-scene synthetic split to the $\sim$ 56,000-image COCO person-keypoint subset. The legacy recipe is the inherited 20-epoch schedule at learning rate 10^{-4}, cosine-decayed to 10^{-6}, weight decay 10^{-4}; the sweep variant searches over fine-tune epochs $\in \{10, 15, 20, 30\}$, learning rate $\in [10^{-5}, 5 \times 10^{-4}]$ (log-uniform), and weight decay $\in \{10^{-5}, 10^{-4}, 10^{-3}\}$ for a total budget of 12 runs with Hyperband early termination at the third validation iteration (epoch 6).

4.6.2 Headline numbers

Configuration	Synth val PGA	COCO val PGA (kNN, GT kp)	COCO E2E PGA (kNN)	W&B run
PG-GAT (synth-only, `champion-arch`)	0.9634	0.9010	[Phase C]	`08d0ffde`
Legacy COCO ft (20 ep, inherited recipe)	0.9178	0.9565	[Phase C]	`ugzbgepn`
Single fine-tune of `champion-arch` (20 ep)	—	0.9695	0.9559	`g0h6pu00`
Fine-tune sweep winner (`meng-headline`)	**0.9634**	**0.9715**	**0.9591**	`b5noyg7t`
HigherHRNet AE (reference upper bound)	—	—	0.9847	—

Table 4.7. COCO fine-tune results. The fine-tune sweep winner is the headline checkpoint and produces the defended COCO numbers cited downstream. The HigherHRNet associative-embedding reference is included for context; the gap between it and the headline E2E number is the detector-independence gap discussed in Section 4.9.

4.6.3 Interpretation

Four observations follow from Table 4.7.

4.6.3.0.1 1. THE FINE-TUNE IS THE SINGLE LARGEST GAIN OF ANY EXPERIMENT IN THIS CHAPTER. COCO kNN PGA lifts from the synth-only PG-GAT's ~ 0.91 to 0.9715 at the fine-tune sweep winner, a gain of approximately 0.06 points. For comparison, the PG-GAT isolation modifications of Section 4.4 produce ~ 0.005 each, the architecture sweep of Section 4.5 produces ~ 0.07, and every other method in this chapter produces a gain below either of these magnitudes. The COCO fine-tune is the largest available lever in the experimental pipeline, which is consistent with the diagnosis from Section 4.1 that the synthetic-to-real distribution shift is the dominant axis of error in COCO performance.

4.6.3.0.2 2. THE SWEEP PRODUCES A MODEST IMPROVEMENT OVER THE LEGACY RECIPE. The single fine-tune of `champion-arch` at the legacy 20-epoch recipe achieves 0.9695 COCO kNN. The 12-run Bayesian sweep around that recipe achieves 0.9715 COCO kNN, an improvement of $+0.0020$. This is statistically below the per-image standard deviation of the metric, so the sweep should not be cited as a substantive improvement. What it does provide is a defensibility argument: the inherited learning rate, weight decay, and epoch count are within a search neighbourhood whose winner was selected by Bayesian optimisation over 12 configurations rather than being inherited unexamined. The sweep winner is promoted to `meng-headline` because the headline benefits

from carrying the strongest available checkpoint, not because the lift over the single-fine-tune is meaningful.

4.6.3.0.3 3. THE SYNTH VAL PGA IS PRESERVED THROUGH FINE-TUNING. A common failure mode of synthetic-to-real fine-tuning is catastrophic forgetting: the network specialises to the target distribution and loses the source-distribution performance. The fine-tune sweep winner retains the same 0.9634 synth val PGA as the architecture-sweep pre-training checkpoint, which means the COCO gain comes at no cost to synthetic-distribution accuracy. The fine-tune is therefore additive rather than substitutional; the legacy variant trained from the older two-layer baseline architecture shows a more pronounced synth–COCO trade-off (0.9178 synth val after COCO ft, against ~ 0.88 for the pre-fine-tune baseline of Section 4.1), which is a known artefact of fine-tuning a smaller backbone for longer on a much larger dataset. The sweep-winner result is the one defended downstream.

4.6.3.0.4 4. THE DETECTOR-INDEPENDENCE GAP CLOSES FROM INVENTORY TO RE-RUN. The gap between the headline E2E result (0.9591) and the HigherHRNet AE reference (0.9847) is 0.0256 in the headline configuration. This is the smallest detector-independence gap measured across the experimental work, and the autonomous-pipeline result of Section 4.9 inherits this checkpoint and reports the operational end-to-end number. The fact that AE retains a small but non-zero advantage even at the fine-tune-sweep winner is consistent with AE's joint-training advantage (the tag head is trained end-to-end with the detector on the COCO distribution), and the gap is the question the discussion of Section 5.3 addresses.

4.7 HYPERBOLIC PG-GAT

This section reports the results for the hyperbolic PG-GAT variant of Section 3.10. Three single-run measurements and one Bayesian sweep are reported, collectively testing the two pre-registered hypotheses fixed in Section 3.10.4: H1, that hyperbolic embedding matches or beats Euclidean at the same output dimension, and H2, that hyperbolic embedding matches Euclidean at a substantially smaller output dimension. The collected evidence supports H2 only in the synthetic-only regime; once COCO fine-tuning is applied, the dim-32 advantage does not persist and the variant is reported honestly as an efficiency trade rather than a quality win.

4.7.1 Setup

Three single-run configurations are reported. H1 trains a hyperbolic PG-GAT at the matched Euclidean output dimension $d = 128$; H2 trains a hyperbolic PG-GAT at the reduced output dimension $d = 32$ ($4\times$ smaller); Option B fine-tunes the H2 checkpoint on COCO for 20 epochs using the legacy fine-tune recipe of Section 4.6. All three use the Lorentz-model output projection and the hyperbolic

contrastive loss of Section 3.10.2, with the tangent-scale τ initialised at $1/\sqrt{d}$ and the margin $\gamma_{\mathbb{H}} = 2.0$. A complementary 8-run Bayesian sweep is reported separately to defend the choice of d against the architecture-sweep search space of Section 4.5.

4.7.2 Headline numbers

Configuration	Synth val PGA	COCO val PGA (kNN, GT kp)	W&B run
PG-GAT Euclidean ($d = 128$, ablation row)	0.8896	0.8868	`r5ekg0zl`
PG-GAT Euclidean (`champion-arch`, $d = 256$)	0.9634	0.9010	`08d0ffde`
PG-GAT Euclidean (`meng-headline`, COCO ft)	0.9634	0.9715	`b5noyg7t`
H1: Hyperbolic, $d = 128$, synth-only	0.8773	[Phase C]	`aox7h3di`
H2: Hyperbolic, $d = 32$, synth-only	0.8856	[Phase C]	`g45tdxi5`
Option B: Hyperbolic, $d = 32$, COCO fine-tuned	0.8900	0.9399	`lragtfzu`
Hyperbolic sweep winner (`champion-hyperbolic`)	0.8795	0.8876	`ua4wdb3p`

Table 4.8. Hyperbolic PG-GAT against the Euclidean Reference points. The hyperbolic sweep winner is reported alongside the single-run ablations as the broader defensibility evidence for the dimension choice. None of the hyperbolic configurations beat the Euclidean fine-tuned headline (0.9715); the dim-32 row matches the dim-128 Euclidean ablation row at 4× smaller embedding, which is the efficiency-trade result.

4.7.3 Interpretation

The hyperbolic configurations populate the table cleanly: H2 (dim-32) is the strongest of the three single-run hyperbolic configurations on synthetic validation; H1 (dim-128) is the weakest; Option B (dim-32 with COCO fine-tuning) is the strongest hyperbolic configuration overall.

4.7.3.0.1 1. H2 SUPPORTS THE EFFICIENCY HYPOTHESIS; H1 DOES NOT SUPPORT THE QUALITY HYPOTHESIS.

H1 (dim-128, 0.8773) is below the Euclidean ablation row at the same architecture (0.8896), which contradicts the quality-hypothesis H1. H2 (dim-32, 0.8856) is within seed-noise of the Euclidean ablation row, which supports the efficiency-hypothesis H2: a 4× smaller hyperbolic embedding matches the larger Euclidean one on the synthetic distribution.

4.7.3.0.2 2. THE EFFICIENCY ADVANTAGE DOES NOT SURVIVE COCO FINE-TUNING.

Option B fine-tunes H2 on COCO and reaches 0.8900 synth val, which is above the H2 starting point but still well below the Euclidean fine-tune sweep winner (0.9634, `meng-headline`). The Euclidean fine-tune effect is large (~ 0.06, Section 4.6); the hyperbolic fine-tune effect at dim-32 is small

(~ 0.004). The asymmetry is the load-bearing finding here: the smaller hyperbolic embedding has less representational headroom to absorb the COCO distribution, and the fine-tune lift is correspondingly smaller. The dim-32 hyperbolic advantage at synthetic-only does not translate into a competitive COCO-fine-tuned result.

4.7.3.0.3 3. THE HYPERBOLIC SWEEP CONFIRMS THE PICTURE. The 8-run Bayesian sweep over hyperbolic output dimension, depth, and hidden dimension (Section 3.13.3, sweep id `qpggbsdn`) selects a configuration whose synth val PGA is 0.8795, which is consistent with the single-run H1 result and below the dim-32 single-run H2 result. Hyperbolic embedding does not produce a configuration that beats the matched Euclidean ablation row in the sweep, which rules out the possibility that the H1 failure was specific to the matched-dimension choice. The sweep also surfaces a more nuanced finding: the synth-to-COCO transfer gap at the hyperbolic sweep winner is approximately $4\times$ smaller than at the matched Euclidean configuration, which is a separate efficiency-style observation about transfer robustness rather than absolute quality.

4.7.3.0.4 4. HONEST FRAMING: EFFICIENCY, NOT QUALITY. The headline summary of this section is therefore: hyperbolic PG-GAT at dim-32 matches the Euclidean ablation row at dim-128 in the synthetic-only regime, which is an efficiency advantage of $4\times$ smaller embeddings; after COCO fine-tuning, the Euclidean configuration takes the lead by ~ 0.08 COCO val PGA; the hyperbolic variant is therefore not a competing answer to the headline question but a documented efficiency observation that may be useful in deployment regimes that prioritise embedding size over absolute COCO PGA. The discussion of why hyperbolic does not win at the full architecture is the subject of Section 5.4.

4.8 TRIGAT — TRIPLET-BASED PRIMITIVE

This section reports the result for the triplet-based PG-GAT variant of Section 3.11. The triplet primitive is tested under the same training schedule and downstream read-out as PG-GAT so that the comparison controls for everything except the per-node primitive. The result is a clean negative: TriGAT underperforms PG-GAT on synthetic validation by margins that exceed seed-noise, and the voting step is identified as the dominant failure mode.

4.8.1 Setup

TriGAT is trained for 50 epochs on the synthetic training split with the optimiser and schedule of Section 3.5.3, matching the PG-GAT training budget exactly. The triplet graph contains one node per detected skeleton triplet, which gives a per-scene node count of approximately $19K$ for a K-person scene with full visibility — comparable in scale to the joint graph's $17K$. At inference, k-means is

applied to the triplet embeddings and the per-triplet cluster labels are voted into per-joint labels by the rule of Section 3.11.3 (pivot triplet weighted at twice the wing triplets, ties broken by the lowest-numbered triplet's cluster). The final per-joint partition is then Hungarian-matched to ground-truth person identifiers for the PGA computation.

4.8.2 Headline numbers

Configuration	Synth val PGA	COCO val PGA (kNN, GT kp)	W&B run
PG-GAT (synth-only, ablation row)	0.8896	[Phase C]	`r5ekg0zl`
TriGAT	0.8740	[Phase C]	`y0tw6vc9`
Δ vs PG-GAT	-0.0156	—	—

Table 4.9. TriGAT against the matched-budget PG-GAT ablation row. The triplet primitive underperforms by 0.0156 on synthetic validation in a single-seed measurement.

4.8.3 Interpretation

The TriGAT result is a clean negative under matched training compute. Three observations:

4.8.3.0.1 1. THE UNDERPERFORMANCE IS STRUCTURAL, NOT UNDER-TRAINED. TriGAT uses the same training schedule as PG-GAT, operates on a graph of comparable scale ($19K$ nodes versus $17K$ for the joint graph), and consumes the same contrastive loss signal. The only material difference is the per-node primitive (a triplet rather than a joint) and the requirement to vote triplet cluster labels back into joint labels at inference. The fact that TriGAT underperforms by 0.0156 at matched compute therefore rules out training budget as the cause and identifies one of these two structural differences as the failure mode.

4.8.3.0.2 2. THE VOTING STEP IS THE DOMINANT FAILURE MODE. Each joint participates in up to three triplets, each of which may be assigned an arbitrary cluster identifier by the per-scene k-means. The voting rule of Section 3.11.3 resolves these per-triplet labels into a single per-joint label by weighted majority, but the per-triplet labels are noisy — a triplet whose pivot is near a same-type collision can be mis-assigned in ways that the voting cannot correct. A single mis-assigned triplet in a person can move two or three of that person's joints to a wrong cluster, which amplifies small per-triplet errors into joint-level mistakes. The voting step is therefore not a benign post-processing step but a load-bearing component whose error mode is the dominant contribution to the negative result. Companion work that read out the triplet embeddings with a decisive per-type matching algorithm records an even larger collapse to approximately 0.80 synth val under the same training conditions,

which is consistent with the same diagnosis: the more decisive the per-triplet decision, the more catastrophic the propagation of a single mis-assigned triplet through the voting rule.

4.8.3.0.3 3. THE TRIPLET PRIMITIVE IS NOT WRONG IN PRINCIPLE. The articulation argument that motivated TriGAT — that two people in similar absolute positions can be distinguished by their joint angles — is empirically valid: preliminary diagnostic measurements on the triplet feature distribution show that pivot angles carry identity-discriminating signal that single-joint position features do not directly access. What this section demonstrates is that the architectural realisation of the triplet primitive (triplet-graph node + per-triplet clustering + voting to joints) gives up more in the voting step than the articulation signal contributes through the embedding. A variant that learns the joint assignment directly from triplet embeddings without an intermediate triplet-clustering step would isolate the articulation signal without the voting noise; that variant would be a substantial re-architecture and is recorded as future work rather than implemented here.

4.9 END-TO-END INTEGRATION WITH HIGHERHRNET

This section closes the experimental loop by reporting end-to-end pose-grouping results in which the keypoint input comes from HigherHRNet-W32-512 [10] rather than from the ground-truth annotation file. Two configurations are reported: the oracle-K configuration in which the number of people K is taken from the ground truth, and the autonomous configuration in which K is predicted by the frozen-backbone K-head of Section 3.12.3. The oracle row isolates the embedding+grouping contribution net of K-estimation error; the autonomous row reports the operational end-to-end accuracy of a system with no ground-truth dependency at inference.

4.9.1 Setup

The detector is HigherHRNet-W32-512 with COCO-trained weights, held frozen throughout (no fine-tuning, no head replacement, no retraining). The detector emits per-joint-type heatmaps; internal non-maximum suppression extracts the keypoint candidate set that both HigherHRNet's own associative-embedding (AE) grouping and the PG-GAT grouping consume. A Hungarian assignment between detected keypoints and ground-truth keypoints (10-pixel matching threshold) attaches a ground-truth person identifier to each matched detection for the metric computation, and the per-detection PGA of Equation (3.17) is computed against this denominator for both AE and PG-GAT. The PG-GAT embedding network is the `meng-headline` checkpoint of Section 4.6. The K-head used in the autonomous configuration is the frozen-backbone K-head trained on COCO embeddings of Section 3.12.3.

4.9.2 Headline numbers

Configuration	COCO E2E PGA (kNN, HRNet det.)	W&B run
HigherHRNet AE (reference upper bound)	0.9847	—
PG-GAT + k-means (`meng-headline`, oracle K)	0.9591	`b5noyg7t`
PG-GAT + k-means (`meng-headline`, predicted K)	[Phase C]	`b5noyg7t`

Table 4.10. End-to-end COCO PGA with HigherHRNet detection. The reference upper bound is HigherHRNet's native AE-grouping output on the same detections; the gap is the detector-independence cost of the modular pipeline.

4.9.3 Interpretation

Three observations close the experimental chapter.

4.9.3.0.1 1. THE DETECTOR-INDEPENDENCE GAP IS SMALL. The headline E2E result is 0.9591 against the HigherHRNet AE reference of 0.9847, a gap of 0.0256. This is the smallest detector-independence gap measured across the experimental work (the pre-sweep pre-fine-tune PG-GAT-full configuration recorded a gap of 0.0301 at the same detector; the legacy 20-epoch COCO fine-tune narrowed it to 0.0225; the fine-tune sweep settles it at 0.0256 against the headline AE row). The remaining 0.0256 is the architectural cost of decoupling detection from grouping: the AE tag head is trained end-to-end with the detector on the COCO distribution and benefits from joint optimisation that the modular PG-GAT pipeline cannot exploit by construction. The question of whether this gap is acceptable is the subject of Section 5.3; the empirical fact recorded here is that it exists and is small.

4.9.3.0.2 2. THE DECOUPLING IS THE CONTRIBUTION. HigherHRNet's 0.9847 is the strongest published bottom-up grouping result on this distribution at this detector configuration. The fact that PG-GAT achieves 0.9591 on the same detections without seeing the detector's weights, training data, or training procedure is the empirical demonstration of the modularity claim: the same embedding network would compose with any other keypoint detector to produce a complete pose-estimation system, and the price of that modularity is 0.0256 on this comparison. This decomposability is the contribution measured here against end-to-end methods such as DEKR [16] and ED-Pose [24] that couple detection and grouping in a single trainable pipeline.

4.9.3.0.3 3. THE AUTONOMOUS PIPELINE IS THE OPERATIONAL SYSTEM. Substituting the predicted K from the frozen-backbone K-head of Section 3.12.3 for the ground-truth K gives the operational version of the same pipeline: HigherHRNet detects keypoints, PG-GAT embeds them, the K-head predicts the person count, and k-means partitions the embeddings. The resulting E2E PGA is reported in the predicted-K row of Table 4.10 and is the number a deployment would actually achieve with no oracle. The K-head error is the additional cost on top of the oracle-K row; the gap between the two rows is the K-estimation error contribution. The autonomous row demonstrates that every component of the pipeline functions end-to-end without any ground-truth dependency at inference, which closes the methodology of Section 3.12.

4.10 MODEL SIZE AND THE MODULARITY CONTRIBUTION

This section accounts for the parameter counts of every component in the pipeline and corrects an interpretation of model size that an early reading of the results invited. The corrected interpretation is recorded here because it changes the framing of the modularity contribution that the rest of the chapter rests on.

4.10.1 Parameter accounting

Component	Parameter count
PG-GAT (`meng-headline`, no visual features)	170,560
PG-GAT v2 network only (visual feature projection + GAT)	197,280
MobileNetV2 backbone (frozen, supplies v2 features)	2,200,000
K-head (frozen-backbone, MLP)	18,625
HigherHRNet AE tag head only	$\sim$ 561
HigherHRNet-W32 (full detector, backbone + heatmap + AE)	28,645,331

Table 4.11. Parameter counts across the pipeline. The PG-GAT network itself is substantially smaller than the HigherHRNet detector; the AE tag head alone is two orders of magnitude smaller than PG-GAT.

4.10.2 Correcting an early framing

An early version of the analysis quoted PG-GAT as "168$\times$ smaller than HigherHRNet" by comparing the PG-GAT parameter count (170,560) against the full HigherHRNet model (28,645,331). This number is technically correct but methodologically misleading. The HigherHRNet AE tag head — the component that PG-GAT replaces — is approximately 561 parameters, which is more than two orders

of magnitude *smaller* than PG-GAT, not larger. The 168×-smaller comparison was therefore between unrelated components (PG-GAT versus a full detector + tag head, rather than PG-GAT versus the tag head alone), and citing it as a size advantage misrepresents what is being compared.

4.10.3 What the contribution actually is

PG-GAT is *larger* than the AE tag head it replaces: 170,560 versus ~ 561, a ratio of $\sim 304\times$ larger. The PG-GAT contribution against AE is therefore not parameter efficiency but modularity. The AE tag head is structurally tied to the HigherHRNet backbone because it is trained end-to-end with the heatmap head on the same intermediate features; it cannot be detached and applied to keypoints produced by any other detector without re-training the entire HigherHRNet pipeline. PG-GAT consumes already-detected keypoint coordinates as its input and is therefore detector-agnostic by construction: any keypoint detector that outputs the COCO 17-joint format can be substituted upstream of PG-GAT without modifying the embedding network. The parameter-count comparison is incidental to this property; the methodological contribution that the chapter has been measuring is the modularity itself, which Section 4.9 reports as a 0.0256 detector-independence gap against an end-to-end-trained alternative.

The honest summary is: PG-GAT pays a parameter-count cost of $\sim 170{,}000$ to replace a 561-parameter AE tag head, and gains the ability to plug into any detector without joint training, at a 0.0256 COCO PGA cost against the joint-training alternative. The trade is whether modularity is worth 0.0256 PGA and 170,000 parameters; the discussion of Section 5.3 argues that it is, for deployment scenarios that require the embedding network to compose with multiple detectors or that require the grouping component to be developed independently of the detection pipeline.

4.11 PG-GAT V2 — VISUAL FEATURE AUGMENTATION

This section reports the result for the visual-feature-augmented PG-GAT variant of Section 3.9. The hypothesis under test is that pre-trained ImageNet features sampled at each keypoint location add identity signal that the positional input of Section 3.3.2 cannot provide. The result is a documented negative: visual features lift COCO grouping by less than one point while adding 2.2×10^6 MobileNet parameters to the pipeline, and the resulting model size exceeds HigherHRNet's while still scoring below it.

4.11.1 Setup

Two training schedules are reported: the 20-epoch initial schedule at learning rate 10^{-3} with cosine decay to 10^{-5}, and the 50-epoch extended schedule that resumes from the 20-epoch checkpoint at a

halved initial learning rate 5×10^{-4} and continues for 30 additional epochs. Both train on the cached COCO MobileNetV2 features of Section 3.9.2 (54,564 training samples, 2,258 validation samples) with the contrastive loss of Equation (3.20) only. The MobileNetV2 backbone is held frozen throughout; the visual-feature projection block of Equation (3.29) is trained jointly with the GAT. Validation runs every two epochs on the cached COCO validation features.

4.11.2 Headline numbers

Configuration	COCO val PGA (kNN, GT kp)	Param count	W&B run
PG-GAT v1 (`meng-headline`, no visual features)	0.9715	170,560	`b5noyg7t`
PG-GAT v2, 20-epoch	0.9731	197,280 + 2.2M MobileNet	`vg43ee4u`
PG-GAT v2, 50-epoch extended	0.9749	197,280 + 2.2M MobileNet	`o8978au7`
HigherHRNet AE (reference upper bound)	—	28,645,331	—

Table 4.12. PG-GAT v2 against the no-visual-features PG-GAT headline and the HigherHRNet AE reference. The v2 lift over the headline is +0.0034 COCO val PGA (50-epoch extended); the MobileNetV2 backbone that produces the visual features contributes a further 2.2×10^6 parameters to the total pipeline size.

4.11.3 Interpretation

Three observations.

4.11.3.0.1 1. THE VISUAL SIGNAL IS REAL BUT SMALL. The 20-epoch v2 number (0.9731) is +0.0016 above the no-visual-features headline (0.9715), and the 50-epoch extended run lifts to 0.9749 for an additional +0.0018 over the 20-epoch result and a total lift of +0.0034 over the headline. Both differences are above the per-image standard deviation of the metric, which means the visual signal is contributing a measurable lift in absolute terms. The 50-epoch extended run rules out the possibility that the v2 architecture is undertrained at 20 epochs; the additional 30 epochs add +0.0018 COCO val PGA, which is statistically detectable but practically negligible.

4.11.3.0.2 2. THE LIFT IS BOUGHT AT SUBSTANTIAL PARAMETER COST. The PG-GAT v2 network itself adds approximately 27,000 parameters over the baseline PG-GAT (the projection block of Equation (3.29) plus the adjusted first GAT layer). The MobileNet backbone that supplies the visual features adds a further 2.2×10^6 parameters, all of which must be present at inference time to compute the visual features for new images. The total deployable pipeline therefore consists of approximately 30.8×10^6 parameters (HigherHRNet detector 28.6×10^6 + MobileNet backbone

2.2×10^6 + PG-GAT v2 network 0.2×10^6), which exceeds HigherHRNet's all-in size of 28.6×10^6 while producing a COCO PGA below HigherHRNet's 0.9847. The size accounting is the subject of Section 4.10; the empirical bottom-line is that the trade is unfavourable.

4.11.3.0.3 3. THE RESULT IS REPORTED AS A NEGATIVE FINDING. The hypothesis under test was that visual features would push the modular PG-GAT pipeline past the HigherHRNet AE reference of 0.9847. The result is 0.9749 at the 50-epoch extended run, which is 0.0098 below the AE reference and only 0.0034 above the no-visual-features PG-GAT. The visual signal exists but is insufficient to close the gap to AE, and the price in parameter count exceeds the price the HigherHRNet AE alternative pays in total pipeline size. On this evidence, visual-feature augmentation is not pursued as a follow-on research direction; the result is reported here in full so that future work that revisits visual augmentation (for example with a larger backbone, or end-to-end joint training of backbone and grouping network) has a clear baseline to beat.

CHAPTER 5 DISCUSSION

This chapter interprets the results of Chapter 4, answers the research questions from Chapter 1, and examines the limitations of the work. The central question of the chapter is *why* the investigated methods succeeded or failed, not merely what the numbers were.

5.1 WHY THE LEARNED GROUPING HEADS FAILED

Discusses the shared failure mode of DEC, slot attention, and graph partitioning: each added a learned assignment mechanism on top of embeddings that were already close to separable under simple kNN. When the embedding is the bottleneck, stacking a learned head reshapes the assignment space but does not improve separation, and can actively hurt transfer (the learned head overfits to synthetic structure).

5.2 WHY SA-DMON FAILED

Root-cause analysis of the ten SA-DMoN experiments. The modularity objective is fundamentally misaligned with pose grouping: the kNN graph encodes *spatial* proximity, but the target communities are defined by *identity*. The spatial null model either cancels with the spatial adjacency or is miscalibrated, and no null-model reformulation (learnable / clamped / median σ, feature-only adjacency, entropy regularisation) repaired the misalignment. This is a structural, not a tuning, problem.

5.3 WHY PG-GAT WORKS

Discusses what each of the three modifications buys, drawing on the ablation numbers from Chapter 4. Type-pair attention lets the network weight joint-type relationships independently. The skeleton-aware positional encoding provides translation/scale invariance that the contrastive loss alone does not enforce. The type-repulsion term addresses the hardest pairs (same-type, different-person keypoints that look locally similar). Together they make the embedding separable enough that kNN works.

5.4 INTERPRETING THE HYPERBOLIC RESULT

Interprets the dim-32 hyperbolic advantage and its disappearance after fine-tuning. Pre-fine-tune, hyperbolic geometry helps because the model is relying on the geometric prior (trees embed naturally in hyperbolic space). After fine-tuning, the Euclidean model learns a sufficient representation from data, and the geometric prior becomes a constraint rather than a help. The result is a quality-matched efficiency trade, not a quality win.

5.5 WHY TRIGAT UNDERPERFORMED

The triplet primitive is semantically richer per-token but introduces a voting step that propagates errors from clusters to joint labels. A single mis-voted triplet cascades across the three joint votes that triplet contributes to, and the cascaded effect dominates the gap between TriGAT and joint-level PG-GAT in the results of Chapter 4.

5.6 COMPARISON WITH HIGHERHRNET AE — THE MODULARITY TRADE-OFF

Discusses the 0.955 vs 0.985 gap. HigherHRNet's AE head is tiny ($\sim$561 parameters) but inseparable from its backbone; it gets quality from the backbone features being trained jointly with the tag. PG-GAT is larger (170K parameters) but detector-agnostic — a capability the AE head cannot offer. The correct comparison is not "who wins on COCO" but "what does each buy." This framing matters for the modularity contribution claim.

5.7 ANSWERS TO THE RESEARCH QUESTIONS

Explicit answers to RQ1, RQ2, RQ3 from Chapter 1, grounded in the numbers from Chapter 4.

5.8 LIMITATIONS AND THREATS TO VALIDITY

Known limitations: reliance on detector-supplied K (K-estimation is outside the MEng core); synthetic-to-real domain gap mitigated but not eliminated; only one backbone evaluated end-to-end; PGA is Hungarian-matched joint accuracy, not mAP, so comparison to COCO-standard HPE numbers is indirect; COCO val2017 is the only real-image benchmark reported.

CHAPTER 6 CONCLUSION

This chapter summarises the contributions of the dissertation, consolidates the answers to the research questions, and indicates directions for future work. The PEng follow-up is the main pointer forward, but several MEng-local directions are also noted.

6.1 SUMMARY OF CONTRIBUTIONS

Restates the five numbered contributions from Section 1.6, each now grounded in the numbers from Chapter 4. PG-GAT is the main technical contribution. The empirical finding that the embedding dominates the grouping head is the main scientific contribution. The COCO fine-tuning protocol, the hyperbolic variant, and the TriGAT negative result are supporting contributions.

6.2 ANSWERS TO THE RESEARCH QUESTIONS

Condensed one-paragraph answers to RQ1 (yes — PG-GAT + kNN reaches 0.957 COCO), RQ2 (all three PG-GAT modifications contribute independently; COCO fine-tuning adds the largest single improvement), and RQ3 (PG-GAT is 0.03 PGA below HigherHRNet AE end-to-end, but is detector-agnostic — the trade-off is modularity for a small quality cost).

6.3 FUTURE WORK

Three directions. (1) The PEng follow-up re-frames grouping as structured assignment and develops SCOT, SCOT-KI, and THA — closing on HigherHRNet's 0.985 with algorithmic rather than embedding changes. (2) A fully hyperbolic PG-GAT architecture (hyperbolic attention, not only hyperbolic output) would test the geometric-prior hypothesis more cleanly than the partial variant evaluated here. (3) End-to-end joint training with a detector would give up modularity to regain the coupling advantage AE exploits, and would quantify how much of the 0.03 gap is fundamental versus avoidable.

REFERENCES

[1] T.-Y. Lin, M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollár, and C. L. Zitnick, "Microsoft COCO: Common objects in context," in *European Conference on Computer Vision (ECCV)*, 2014.

[2] Y. Chen, Y. Tian, and M. He, "Monocular human pose estimation: A survey of deep learning-based methods," *Computer Vision and Image Understanding*, vol. 192, p. 102897, 2020.

[3] C. Zheng, W. Wu, C. Chen, T. Yang, S. Zhu, J. Shen, N. Kehtarnavaz, and M. Shah, "Deep learning-based human pose estimation: A survey," *ACM Computing Surveys*, vol. 56, no. 1, pp. 1–37, 2023.

[4] M. Andriluka, L. Pishchulin, P. Gehler, and B. Schiele, "2D human pose estimation: New benchmark and state of the art analysis," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2014, pp. 3686–3693.

[5] A. Toshev and C. Szegedy, "Deeppose: Human pose estimation via deep neural networks," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2014, pp. 1653–1660.

[6] S.-E. Wei, V. Ramakrishna, T. Kanade, and Y. Sheikh, "Convolutional pose machines," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2016, pp. 4724–4732.

[7] A. Newell, K. Yang, and J. Deng, "Stacked hourglass networks for human pose estimation," in *European Conference on Computer Vision*. Springer, 2016, pp. 483–499.

[8] B. Xiao, H. Wu, and Y. Wei, "Simple baselines for human pose estimation and tracking," in *European Conference on Computer Vision*, 2018, pp. 466–481.

[9] K. Sun, B. Xiao, D. Liu, and J. Wang, "Deep high-resolution representation learning for human pose estimation," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019.

[10] B. Cheng, B. Xiao, J. Wang, H. Shi, T. S. Huang, and L. Zhang, "HigherHRNet: Scale-aware representation learning for bottom-up human pose estimation," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020.

[11] K. He, G. Gkioxari, P. Dollár, and R. Girshick, "Mask R-CNN," in *Proceedings of the IEEE International Conference on Computer Vision*, 2017, pp. 2961–2969.

[12] H.-S. Fang, S. Xie, Y.-W. Tai, and C. Lu, "RMPE: Regional multi-person pose estimation," in *Proceedings of the IEEE International Conference on Computer Vision*, 2017, pp. 2334–2343.

[13] Z. Cao, T. Simon, S.-E. Wei, and Y. Sheikh, "Realtime multi-person 2D pose estimation using part affinity fields," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017.

[14] S. Kreiss, L. Bertoni, and A. Alahi, "PifPaf: Composite fields for human pose estimation," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2019, pp. 11 977–11 986.

[15] A. Newell, Z. Huang, and J. Deng, "Associative embedding: End-to-end learning for joint detection and grouping," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[16] Z. Geng, K. Sun, B. Xiao, Z. Zhang, and J. Wang, "Bottom-up human pose estimation via disentangled keypoint regression," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021.

[17] G. Brasó, N. Kister, and L. Leal-Taixé, "The center of attention: Center-keypoint grouping via attention for multi-person pose estimation," in *IEEE International Conference on Computer Vision (ICCV)*, 2021, pp. 11 853–11 863.

[18] Y. Li, S. Zhang, Z. Wang, S. Yang, W. Yang, S.-T. Xia, and E. Zhou, "TokenPose: Learning keypoint tokens for human pose estimation," in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2021, pp. 11 313–11 322.

[19] Y. Yuan, R. Fu, L. Huang, W. Lin, C. Zhang, X. Chen, and J. Wang, "HRFormer: High-resolution transformer for dense prediction," in *Advances in Neural Information Processing Systems*, vol. 34, 2021, pp. 7281–7293.

[20] Y. Xu, J. Zhang, Q. Zhang, and D. Tao, "ViTPose: Simple vision transformer baselines for human pose estimation," in *Advances in Neural Information Processing Systems*, vol. 35, 2022, pp. 38 571–38 584.

[21] ——, "ViTPose++: Vision transformer for generic body pose estimation," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 46, no. 2, pp. 1212–1230, 2023.

[22] D. Shi, X. Wei, L. Li, Y. Ren, and W. Tan, "End-to-end multi-person pose estimation with transformers," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022, pp. 11 069–11 078.

[23] Y. Xiao, K. Su, X. Wang, D. Yu, L. Jin, M. He, and Z. Yuan, "QueryPose: Sparse multi-person pose regression via spatial-aware part-level query," in *Advances in Neural Information Processing Systems*, 2022.

[24] J. Yang, A. Zeng, F. Li, S. Liu, R. Zhang, and L. Zhang, "Explicit box detection unifies end-to-end multi-person pose estimation," in *International Conference on Learning Representations (ICLR)*,

2023.

[25] H. Liu, Q. Chen, Z. Tan, J. Liu, J. Wang, X. Su, X. Li, K. Yao, J. Han, E. Ding *et al.*, "Group pose: A simple baseline for end-to-end multi-person pose estimation," in *IEEE International Conference on Computer Vision (ICCV)*, 2023.

[26] X. Sun, B. Xiao, F. Liu, and Y. Wei, "Integral human pose regression," in *European Conference on Computer Vision*, 2018, pp. 529–545.

[27] A. Nibali, Z. He, S. Morgan, and L. Prendergast, "Numerical coordinate regression with convolutional neural networks," *arXiv preprint arXiv:1801.07372*, 2018, arXiv preprint, not peer-reviewed.

[28] J. Li, S. Bian, A. Zeng, C. Wang, B. Pang, W. Liu, and C. Lu, "Human pose regression with residual log-likelihood estimation," in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2021, pp. 11 025–11 034.

[29] Y. Li, S. Yang, P. Liu, S. Zhang, Y. Wang, Z. Wang, W. Yang, and S.-T. Xia, "SimCC: A simple coordinate classification perspective for human pose estimation," in *European Conference on Computer Vision*, 2022, pp. 89–106.

[30] T. Jiang, P. Lu, L. Zhang, N. Ma, R. Han, C. Lyu, Y. Li, and K. Chen, "RTMPose: Real-time multi-person pose estimation based on MMPose," *arXiv preprint arXiv:2303.07399*, 2023, arXiv tech report from OpenMMLab.

[31] P. Lu, T. Jiang, Y. Li, X. Li, K. Chen, and W. Yang, "RTMO: Towards high-performance one-stage real-time multi-person pose estimation," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2024, pp. 1491–1500.

[32] T. Jiang, X. Xie, and Y. Li, "RTMW: Real-time multi-person 2d and 3d whole-body pose estimation," *arXiv preprint arXiv:2407.08634*, 2024, arXiv tech report from OpenMMLab.

[33] R. Khirodkar, T. Bagautdinov, J. Martinez, S. Zhaoen, A. James, P. Selednik, S. Anderson, and S. Saito, "Sapiens: Foundation for human vision models," in *European Conference on Computer Vision*, 2024.

[34] J. Yang, A. Zeng, R. Zhang, and L. Zhang, "X-Pose: Detecting any keypoints," in *European Conference on Computer Vision*, 2024.

[35] Z. Geng, C. Wang, Y. Wei, Z. Liu, H. Li, and H. Hu, "Human pose as compositional tokens," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2023, pp. 660–671.

[36] J. Li, C. Wang, H. Zhu, Y. Mao, H.-S. Fang, and C. Lu, "CrowdPose: Efficient crowded scenes pose estimation and a new benchmark," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2019, pp. 10 863–10 872.

[37] S.-H. Zhang, R. Li, X. Dong, P. Rosin, Z. Cai, X. Han, D. Yang, H. Huang, and S.-M. Hu, "Pose2Seg: Detection free human instance segmentation," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2019, pp. 889–898.

[38] Y. Xiu, J. Li, H. Wang, Y. Fang, and C. Lu, "Pose flow: Efficient online pose tracking," in *British Machine Vision Conference*, 2018.

[39] G. Ning, J. Pei, and H. Huang, "LightTrack: A generic framework for online top-down human pose tracking," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops*, 2020, pp. 1034–1035.

[40] M. Andriluka, U. Iqbal, E. Insafutdinov, L. Pishchulin, A. Milan, J. Gall, and B. Schiele, "PoseTrack: A benchmark for human pose estimation and tracking," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2018, pp. 5167–5176.

[41] S. Jin, W. Liu, E. Xie, W. Wang, C. Qian, W. Ouyang, and P. Luo, "Differentiable hierarchical graph grouping for multi-person pose estimation," in *European Conference on Computer Vision (ECCV)*, 2020.

[42] M. Maher, R. Fathalla, and M. Shaheen, "Graph edge classification for keypoint grouping in multi-person pose estimation," *Journal of Image and Graphics*, vol. 13, no. 2, pp. 130–139, 2025.

[43] S. Yang, Z. Feng, Z. Wang, Y. Li, S. Zhang, Z. Quan, S.-T. Xia, and W. Yang, "Detecting and grouping keypoints for multi-person pose estimation using instance-aware attention," *Pattern Recognition*, vol. 136, p. 109232, 2023.

[44] J. Gilmer, S. S. Schoenholz, P. F. Riley, O. Vinyals, and G. E. Dahl, "Neural message passing for quantum chemistry," in *International Conference on Machine Learning (ICML)*, 2017.

[45] W. L. Hamilton, R. Ying, and J. Leskovec, "Inductive representation learning on large graphs," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[46] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," in *International Conference on Learning Representations (ICLR)*, 2017.

[47] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, "Graph attention networks," in *International Conference on Learning Representations (ICLR)*, 2018.

[48] S. Brody, U. Alon, and E. Yahav, "How attentive are graph attention networks?" in *International Conference on Learning Representations (ICLR)*, 2022.

[49] J. MacQueen, "Some methods for classification and analysis of multivariate observations," in *Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability*, vol. 1, 1967, pp. 281–297.

[50] S. P. Lloyd, "Least squares quantization in PCM," *IEEE Transactions on Information Theory*, vol. 28, no. 2, pp. 129–137, 1982.

[51] D. Arthur and S. Vassilvitskii, "k-means++: The advantages of careful seeding," in *Proceedings of the Eighteenth Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*, 2007, pp. 1027–1035.

[52] K. Wagstaff, C. Cardie, S. Rogers, and S. Schrödl, "Constrained k-means clustering with background knowledge," in *International Conference on Machine Learning (ICML)*, 2001, pp. 577–584.

[53] J. Xie, R. Girshick, and A. Farhadi, "Unsupervised deep embedding for clustering analysis," in *International Conference on Machine Learning (ICML)*, 2016.

[54] L. van der Maaten and G. Hinton, "Visualizing data using t-SNE," *Journal of Machine Learning Research*, vol. 9, pp. 2579–2605, 2008.

[55] F. Locatello, D. Weissenborn, T. Unterthiner, A. Mahendran, G. Heigold, J. Uszkoreit, A. Dosovitskiy, and T. Kipf, "Object-centric learning with slot attention," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[56] K. Cho, B. van Merriënboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk, and Y. Bengio, "Learning phrase representations using RNN encoder-decoder for statistical machine translation," in *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2014, pp. 1724–1734.

[57] J. Shi and J. Malik, "Normalized cuts and image segmentation," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 22, no. 8, pp. 888–905, 2000.

[58] U. Von Luxburg, "A tutorial on spectral clustering," *Statistics and Computing*, vol. 17, no. 4, pp. 395–416, 2007.

[59] L. Yang, D. Chen, X. Zhan, R. Zhao, C. C. Loy, and D. Lin, "Learning to cluster faces via confidence and connectivity estimation," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020.

[60] M. E. J. Newman, "Modularity and community structure in networks," *Proceedings of the National Academy of Sciences*, vol. 103, no. 23, pp. 8577–8582, 2006.

[61] A. Tsitsulin, J. Palowitch, B. Perozzi, and E. Müller, "Graph clustering with graph neural networks," *Journal of Machine Learning Research*, vol. 24, no. 127, pp. 1–21, 2023.

[62] H. W. Kuhn, "The Hungarian method for the assignment problem," *Naval Research Logistics Quarterly*, vol. 2, no. 1–2, pp. 83–97, 1955.

[63] Blender Online Community, "Blender — a 3d modelling and rendering package," http://www.blender.org, Blender Foundation, 2024.

[64] R. Ranftl, K. Lasinger, D. Hafner, K. Schindler, and V. Koltun, "Towards robust monocular depth estimation: Mixing datasets for zero-shot cross-dataset transfer," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 44, no. 3, pp. 1623–1637, 2022.

[65] M. Fey and J. E. Lenssen, "Fast graph representation learning with PyTorch Geometric," in *ICLR Workshop on Representation Learning on Graphs and Manifolds*, 2019.

[66] S. Chopra, R. Hadsell, and Y. LeCun, "Learning a similarity metric discriminatively, with application to face verification," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2005.

[67] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in *International Conference on Learning Representations (ICLR)*, 2019.

[68] ——, "SGDR: Stochastic gradient descent with warm restarts," in *International Conference on Learning Representations (ICLR)*, 2017.

[69] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and É. Duchesnay, "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[70] L. Li, K. Jamieson, G. DeSalvo, A. Rostamizadeh, and A. Talwalkar, "Hyperband: A novel bandit-based approach to hyperparameter optimization," *Journal of Machine Learning Research*,

vol. 18, no. 185, pp. 1–52, 2018.

[71] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "MobileNetV2: Inverted residuals and linear bottlenecks," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 4510–4520.

[72] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, "ImageNet: A large-scale hierarchical image database," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2009, pp. 248–255.

[73] J. L. Ba, J. R. Kiros, and G. E. Hinton, "Layer normalization," *arXiv preprint arXiv:1607.06450*, 2016.

[74] D. Hendrycks and K. Gimpel, "Gaussian error linear units (GELUs)," *arXiv preprint arXiv:1606.08415*, 2016.

[75] R. Sarkar, "Low distortion delaunay embedding of trees in hyperbolic plane," in *Graph Drawing (GD)*, 2011, pp. 355–366.

[76] F. Sala, C. De Sa, A. Gu, and C. Ré, "Representation tradeoffs for hyperbolic embeddings," in *International Conference on Machine Learning (ICML)*, 2018, pp. 4460–4469.

[77] M. Nickel and D. Kiela, "Learning continuous hierarchies in the Lorentz model of hyperbolic geometry," in *International Conference on Machine Learning (ICML)*, 2018, pp. 3776–3785.

[78] M. Kochurov, R. Karimov, and S. Kozlukov, "Geoopt: Riemannian optimization in PyTorch," *arXiv preprint arXiv:2005.02819*, 2020.

[79] L. Biewald, "Experiment tracking with Weights and Biases," https://www.wandb.com/, 2020, software.

APPENDIX A DERIVATION OF HIGHER ORDER MODELS

A.1 EXAMPLE HEADING

A.1.1 Subheading example

Filler text to test template. Filler text to test template.

Filler text to test template. Filler text

to test template. Filler text to test template.

Filler text to test template. Filler text to test template.